

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПРОЕКТИРАНЕ И РАЗРАБОТВАНЕ НА
Compile-time Object-Relational Mapping
библиотека на C++**

ДИПЛОМНА РАБОТА

Изготвил:

Алекс Цветанов, фак. № 121222225

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

доц. д-р инж. Галя Павлова

София, 2026

СЪДЪРЖАНИЕ

I.	Въведение	3
1.1.	Постановка на проблема	3
1.2.	Цел и задачи на разработката	4
1.3.	Обхват и ограничения	5
1.4.	Структура на изложението	5
II.	Преглед на съществуващите решения	7
2.1.	Класификация на подходите	7
2.2.	Критерии за сравнение	9
2.3.	Native клиентски библиотеки	10
2.4.	Асинхронност в съществуващите клиентски стекове	11
2.5.	Runtime query builder-и	11
2.6.	Типизирани SQL DSL решения	11
2.7.	Класически ORM рамки и кодогенерация	12
2.8.	Граници на релационно центрираните абстракции	12
2.9.	Позициониране на настоящата разработка	13
2.10.	Изводи от сравнителния анализ	14
III.	Теоретични основи на Compile-time Object-Relational Mapping и моделите за съхранение	15
3.1.	Таксономия на problem domain-a	15
3.2.	Compile-time Object-Relational Mapping като архитектурен подход	16
3.3.	Статичен модел на данните в C++	17
3.4.	Типизиран query IR	19
3.5.	Теория на връзките и стратегии за съхранение	21
3.6.	Релационен модел	22
3.7.	Документен модел	22
3.8.	Key-value модел	22
3.9.	Графов модел	23
3.10.	Wide-column модел	23
3.11.	Тестова опора на теоретичните твърдения	24

3.12. Граници на унифицираната абстракция	25
IV. Теория на корутините и асинхронното изпълнение в C++	26
4.1. Процеси, нишки, задачи и корутини	26
4.2. Синтаксис, awaitable protocol и езикови binding-и	27
4.3. Корутината като state machine	28
4.4. Coroutine frame, promise object, continuation и безопасност	29
4.5. Корутини, thread pool и event loop инфраструктура	30
4.6. Значение за Compile-time Object-Relational Mapping библиотеката	31
V. Compile-time ORM архитектура	32
5.1. Архитектурни цели и ограничения	32
5.2. Слоеста организация и разделяне на отговорностите	33
5.3. Entity слой като архитектурна основа	34
5.4. Query IR като централен архитектурен компонент	35
5.5. Конекторен слой и формалният contract	35
5.6. Потребителският фасаден слой db<DB>	37
5.7. Миграции като архитектурно разширение	38
5.8. Реализация на основните подсистеми	38
5.8.1. Мини казус с типовете User и Post	39
5.8.2. Скаларни свойства и именуване на контейнерите	40
5.8.3. Field wrappers, правила и placeholder-и	40
5.8.4. Изграждане на query обекти	41
5.8.5. Изпълнение, резултати и достъп до полета	42
5.8.6. Prepared queries като повторно използваем execution артефакт	42
5.8.7. Миграционен слой и DDL генериране	43
5.8.8. Нишкова безопасност и transaction helpers	45
5.8.9. Извод за реализацията	46
5.9. Архитектурни инварианти	47
5.10. Архитектурен извод	47
VI. Асинхронна архитектура на изпълнението	48
6.1. Архитектурна позиция на async слоя	48
6.2. Task<T> като носител на coroutine lifecycle	49

6.3. ThreadPool и run_on_pool(): универсалният execution path	49
6.4. async_db<DB> и capability-gated dispatch	50
6.5. IoContext и native non-blocking integration	53
6.6. Coroutine-aware connection pooling и transaction helpers	53
6.7. Архитектурен извод за async слоя	55
VII. Backend изпълнителни модели и native client библиотеки	57
7.1. Методика на анализа	57
7.2. Релационен базов сценарий: SQLite	59
7.3. Релационна преносимост: PostgreSQL и MySQL	59
7.4. Документен сценарий: MongoDB	60
7.5. Key-value сценарий: Redis	60
7.6. Графов сценарий: Neo4j	61
7.7. Wide-column сценарий: Cassandra	61
7.8. Синтезирана capability матрица	62
7.9. Съпоставка между test evidence и backend contract	65
7.10. Сравнителен синтез	66
VIII. Верификация и емпирична оценка	67
8.1. Стратегия на верификацията	67
8.2. Каталог на unit и integration доказателствата	68
8.3. Карта на зрелостта според доказателствената опора	71
8.4. Методология на benchmark оценката	74
8.5. Основни throughput резултати	75
8.6. Нулева waiting латентност (0ns) и coroutine scheduling overhead	77
8.7. Microbenchmark интерпретация на overhead-a	78
8.8. Ограничения на емпиричната оценка	79
IX. Надграждаемост чрез нови конектори	80
9.1. Формален contract на конекторния слой	80
9.2. Задължителни елементи на минимален конектор	81
9.3. Типизиране, signatures и именуване	82
9.4. Capability механизъм и compile-time ограничения	83
9.5. Минимален connector skeleton и operational connector skeleton	83

9.6. Транзакции, нишкова безопасност и prepared queries	85
9.7. Миграции и schema-aware интеграция	85
9.8. Методика за добавяне на нов backend	86
9.9. Критерии за „напълно имплементиран и функциониращ“ конектор	87
9.10. Верификация чрез тестове	88
9.11. Оценка на наличните конектори	88
X. Ограничения и бъдещо развитие	89
10.1. Стратегически направления за развитие	89
10.2. Разширяване на live backend покритието	90
10.3. Пълна schema-aware интеграция и runtime introspection	91
10.4. Нишкова безопасност и жизнен цикъл на връзките	92
10.5. Ниско ниво на изпълнение и wire protocol оптимизации	92
10.6. Интеграция с C++26 reflection и допълнителна автоматизация	93
10.7. Разширяване на connector contract-a и capability таксономията	93
10.8. Необходимост от по-строга емпирична оценка	94
10.9. Обобщение	95
XI. Заключение	96
11.1. Обобщение на решената задача	96
11.2. Обобщение на постигнатите резултати	97
11.3. Научни и приложни приноси	98
11.4. Основни ограничения и граници на обобщението	99
11.5. Значение за практиката и за бъдещите изследвания	100
11.6. Заключителни бележки	100

РЕЗЮМЕ

Настоящата дипломна работа разглежда проектирането, реализацията и оценяването на Compile-time Object-Relational Mapping библиотека за C++23 [12]. Работата е организирана около две допълващи се оси. Първата е compile-time моделирането на данните и заявките, при което логическата структура на достъпа до данни не се описва чрез низове, интерпретирани на етап изпълнение, а чрез статично типизирани `constexpr` конструкции. Втората е coroutine-базираната асинхронна архитектура, чрез която същият типове ограничен модел се разширява към неблокиращо или псевдо-неблокиращо изпълнение според възможностите на конкретния backend драйвер.

Разработената библиотека използва статичен модел на данните, изразен в C++ чрез конструкции като `property<T, Name>`, `relationship<...>` и `table_name_trait<T>`, както и типизирано междинно представяне на заявките, реализирано чрез `select_query`, `insert_query`, `update_query` и `delete_query`. Този логически слой е независим от конкретния тип база данни. Материализацията към релационни и нерелационни системи се осъществява чрез специализации на `connector_trait<DB>`, които декларират поддържаните възможности и превеждат една и съща логическа заявка към Structured Query Language (SQL), Binary JSON (BSON)-подобни филтри, Redis команди, Cypher или Cassandra Query Language (CQL) според избрания backend.

Съществено разширение на архитектурата е асинхронният слой, изграден върху корутини, `Task<T>`, `ThreadPool`, `async_db<DB>` и coroutine-aware управление на връзки. Показано е, че единна асинхронна абстракция може да бъде реализирана по два начина: чрез offload на блокиращи операции към thread pool и чрез native неблокиращи интерфейси за драйвери, които ги предоставят. Така работата не само описва логическа ORM абстракция, но и изследва изпълнителния модел, чрез който тази абстракция остава practically usable при конкурентни натоварвания.

В дипломната работа се анализират както релационните сценарии за SQLite, PostgreSQL и MySQL, така и нерелационните сценарии за MongoDB, Redis, Neo4j и Cassandra. Показано е как един и същ C++ модел диктува логическата схема, но се материализира различно като таблица, документ, ключ-стойност структура, графов възел или wide-column ред. Отделно се разглеждат native client библиотеките и техните execution semantics, за да се определи къде библиотеката може честно да предложи native асинхронно

поведение и къде правилният избор остава синхронен драйвер с контролирано offload изпълнение.

Верификацията на разработката се основава на unit и integration тестове от хранилището, а емпиричната оценка се допълва от benchmark сценарии за сравнение между синхронния и асинхронния режим. Те покриват изграждането на заявките, scalability-базираното ограничаване на операции, подготвените заявки, миграциите, нишковата безопасност, coroutine инфраструктурата и поведението на конкретните конектори при работа с реални или mock драйвери. Това позволява формулиране не само на архитектурен модел, но и на конкретни критерии за „напълно имплементиран и функциониращ“ конектор и за practically justified асинхронен слой.

Основният принос на дипломната работа е в четири направления. Първо, предложен е practically usable модел за Compile-time Object-Relational Mapping библиотека, който съчетава статична типова безопасност и backend-независима архитектура. Второ, показано е как C++ описанието на данните може да служи като първичен източник за логическа схема при различни типове бази данни. Трето, изградена е coroutine-базирана асинхронна архитектура, която разширява connector модела без компромис с compile-time ограниченията. Четвърто, формализиран е connector contract, чрез който библиотеката може да се надгражда с нови backend реализации и асинхронни възможности без промяна в user-facing query API.

Ключови думи: C++23, Compile-time Object-Relational Mapping, статична типова безопасност, корутини, асинхронно изпълнение, connector_trait, релационни и нерелационни бази данни, емпирична оценка

I. ВЪВЕДЕНИЕ

1.1. Постановка на проблема

Достъпът до данни остава едно от местата, в които разминаването между езика на приложението и модела на съхранение създава най-много структурни грешки. В традиционните слоеве за достъп до бази данни заявките се описват като низове, които се валидират едва при изпълнение. Това означава, че неправилни имена на колони, несъвместими типове, невалидни JOIN конструкции или неправилно подадени параметри се откриват късно — често върху реална база данни и след допълнителни ръчни тестове. Същевременно смяната на backend от релационен към документен, графов или key-value модел обикновено изисква пълно пренаписване на слоя за достъп до данни.

Обектно-релационното картографиране е класическият подход за свързване на обектния модел на приложението с персистентното съхранение [4]. Повечето ORM реализации обаче остават силно зависими от runtime механизми: SQL низове, кодогенерация, анотации, макроси или динамични описания на схема. В контекста на C++ това създава допълнителен проблем. Езикът предлага мощна система от шаблони, constexpr изчисления и concepts, но тези механизми често не се използват докрай в библиотеките за достъп до данни. Настоящата работа изследва доколко е възможно компилаторът да поеме ролята на първи валидатор на query структурата и на backend-съвместимостта.

Съществена особеност на разработваната система е, че въпреки името Compile-time Object-Relational Mapping библиотека, архитектурата ѝ не е ограничена само до релационния модел. Напротив, тя използва еднакъв C++ модел на данните и еднакъв query IR като логически слой над релационни и нерелационни системи. Това поставя втори изследователски въпрос: кои части на ORM абстракцията са универсални и кои неизбежно трябва да се ограничават чрез capability-базиран backend contract.

Третият проблемен пласт е свързан с начина на изпълнение. Дори когато логическият модел и query IR са типове коректни, практическата стойност на една съвременна библиотека за достъп до данни зависи и от това дали тя може да интегрира асинхронно изпълнение без разпадане на типовете гаранции и без принудително преминаване към callback-базиран стил. Така дипломната работа се организира около две допълващи се оси: compile-time моделиране и асинхронна архитектура на изпълнението.

1.2. Цел и задачи на разработката

Основната цел на дипломната работа е да представи проектирането и реализацията на Compile-time Object-Relational Mapping библиотека за C++, която:

1. моделира данните и заявките чрез статично типизирани C++ конструкции;
2. позволява изграждане на SELECT, INSERT, UPDATE и DELETE заявки като constexpr обекти;
3. отделя логическата структура на заявката от backend-специфичното ѝ материализиране;
4. поддържа релационни и нерелационни конектори чрез connector_trait<DB>;
5. използва compile-time capability механизъм за ограничаване на неподдържани операции;
6. демонстрира как C++ моделът определя логическата схема и физическата структура на данните при различни backend-и;
7. реализира coroutine-базиран async слой, който надгражда синхронния connector модел по формален и типово безопасен начин;
8. формализира contract за добавяне на нови конектори, включително когато те поддържат native асинхронно изпълнение;
9. валидира архитектурата чрез систематични unit и integration тестове и чрез емпирична benchmark оценка.

За постигане на тази цел работата решава следните задачи: анализ на съществуващи решения; дефиниране на статичен entity модел; изграждане на query IR; проектиране на connector layer и capability tags; изграждане на coroutine-базиран async слой; имплементиране на конектори за различни типове бази данни и различни execution модели; изследване на миграции, подготвени заявки, нишкова безопасност и async управление на връзки; формулиране на критерии за разширяемост; изграждане на benchmarking методология за оценка на синхронния и асинхронния режим.

1.3. Обхват и ограничения

Работата разглежда библиотеката като слой за моделиране, изграждане и изпълнение на заявки. В обхвата ѝ попадат entity моделът, query IR, placeholder механизмът, подготвените заявки, миграционният модел, connector trait архитектурата, capability ограничаването, coroutine-базираният async слой и конкретните backend реализации в хранилището. Съществена част от оценяването е сценарийният анализ на SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra, както и benchmarking оценката на синхронния и асинхронния execution режим.

Извън непосредствения обхват остават производствено ниво оптимизации като разпределено управление на връзки, автоматизирани migration стратегии за всички backend-и, live benchmarking върху пълна матрица от production среди и пълна експлоатация на бъдещата reflection функционалност на C++26. Тези направления са разгледани като перспектива в Глава X.

1.4. Структура на изложението

Дипломната работа е организирана по следния начин.

Глава II разглежда съществуващи C++ решения за достъп до бази данни и ORM подходи и позиционира разработката спрямо тях.

Глава III представя теоретичните основи на Compile-time Object-Relational Mapping и моделите за съхранение, необходими за по-късния анализ на релационни и нерелационни сценарии.

Глава IV въвежда теоретичния апарат на корутините и асинхронното изпълнение в C++, който е необходим за разбирането на втория основен принос на библиотеката.

Глава V описва compile-time ORM архитектурата на библиотеката: entity описанията, query IR, connector_trait, capability механизмите и мястото на миграциите в общия модел.

Глава VI представя асинхронната архитектура на изпълнението и показва как coroutine-базираният слой се свързва със синхронния connector API, с thread pool модела и с управлението на връзките.

Глава VII анализира как една и съща логическа абстракция и различни execution стратегии се материализират в релационни, документни, key-value, графови и wide-column

backend-и.

Глава VIII обединява верификационните доказателства и емпиричната оценка чрез benchmark резултатите.

Глава IX формализира contract-а за надграждане чрез нови конектори, включително capability и async изискванията.

Глава X очертава ограниченията на текущата версия и възможните направления за бъдещо развитие, а **Глава XI** обобщава резултатите и приносите.

II. ПРЕГЛЕД НА СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ

Анализът на съществуващите решения е необходим, за да се оцени дали предложената архитектура действително допринася с нова стойност, или само преформулира вече познати практики. При C++ библиотеките за достъп до данни могат да се разграничат две основни оси и една допълнителна изпълнителна перспектива: нивото на абстракция, моментът, в който се изгражда и валидира заявката, и моделът на изпълнение. Нивото на абстракция варира от почти директно използване на клиентски C API до пълно ORM поведение. Моментът на валидиране може да бъде runtime, hybrid или compile-time. Изпълнителният модел варира от директни блокиращи API повиквания до backend-специфични асинхронни state machine интерфейси и по-високи слоеве, които се опитват да ги уеднаквят.

Тези оси са особено важни в контекста на настоящата дипломна работа. Ако се наблюдава само “удобството на API-то”, лесно може да се пропусне съществената разлика между библиотека, която просто прави SQL низове по-четими, и библиотека, която действително прехвърля структурната коректност към компилатора. Ако, от друга страна, се гледа само кога възниква грешката, може да се подцени цената на различните нива на абстракция, build toolchain сложност и пригодност за работа с повече от един storage модел. А ако се игнорира изпълнителният модел, лесно се стига до погрешното впечатление, че “асинхронност” е еднотипна способност, независимо дали става дума за native неблокиращо API, callback слой върху event loop или просто за offload към thread pool.

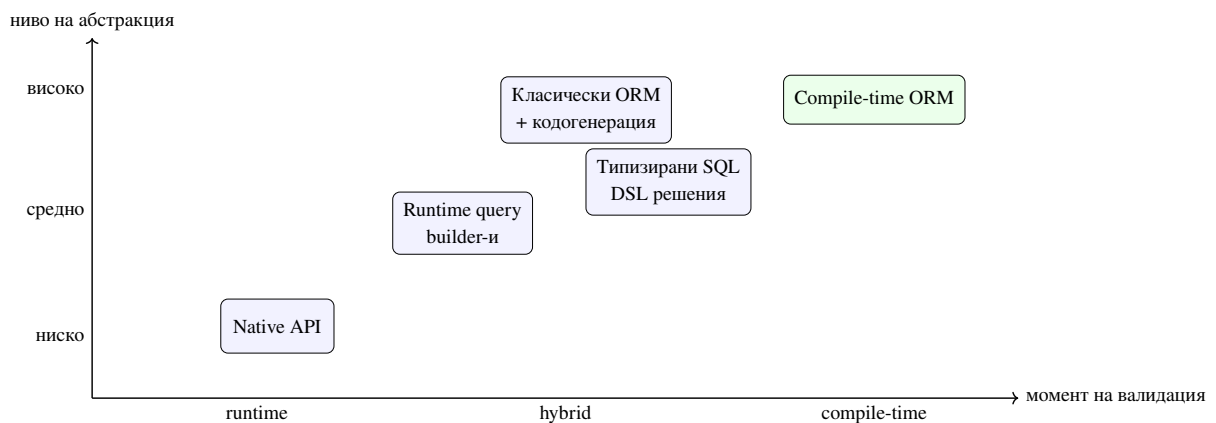
2.1. Класификация на подходите

В най-ниския слой стоят native клиентските библиотеки като SQLiteAPI [7], libpq [18] и MySQLConnector/C [15]. Те предоставят максимален контрол върху заявките, параметрите и резултатите, но не предлагат логическа абстракция. Приложението е принудено да работи директно със SQL низове, индекси на колони, позиции на параметри и backend-специфични типове.

Следващата група са runtime query builder библиотеките. Те обикновено подобряват четимостта на заявките и намаляват част от шаблонния код, но продължават да разчитат на runtime изграждане на текстово представяне. Типовата безопасност е частична и

обикновено спира до границата на API повикването. Структурната валидност на заявката остава проблем на изпълнението.

Най-високото ниво на абстракция е ORM подходът. Там C++ класовете се асоциират с таблици, колони и връзки, а библиотеката генерира необходимите заявки. Предимството е намаляване на boilerplate кода. Недостатъкът е, че често се въвеждат външни генератори, макроси, анотации и oracle runtime метаданни, които затрудняват проследимостта между C++ кода и реалната заявка.



Фигура 1. Позициониране на основните семейства решения според абстракция и момент на валидация

Фигура 1 показва, че compile-time ORM подходът не е просто “още една ORM библиотека”. Той стои в горния десен квадрант, където високото ниво на абстракция се комбинира с ранна структурна проверка. Тази комбинация е рядка, защото изисква повече архитектурна дисциплина в самата библиотека.

Таблица 1. Съпоставка на основните подходи за достъп до данни в C++

Подход	Ниво на абстракция	Валидация	Предимство	Основен недостатък
Native API	ниско	runtime	пълен контрол и близост до драйвера	силна обвързаност с конкретен backend
Runtime query builder	средно	основно runtime	по-четим API и по-малко boilerplate	заявката пак се материализира като низ
Типизиран SQL DSL	средно към високо	частично compile-time	по-добра статична структура за SQL света	обикновено остава ограничен до SQL
Класически ORM	високо	runtime или чрез кодогенерация	удобно object mapping поведение	сложна toolchain интеграция и непрозрачност
Compile-time ORM	високо	compile-time за структурата	ранно откриване на структурни грешки и формален connector contract	по-висока сложност на шаблонния слой

2.2. Критерии за сравнение

За да бъде сравнението академично пълно, не е достатъчно да се каже, че една библиотека е “по-удобна”, а друга е “по-ниско ниво”. В настоящата работа се използват шест сравнителни критерия:

1. **Ниво на логическа абстракция** — доколко библиотеката позволява моделът на данните да се описва независимо от непосредствения синтаксис на backend-а.
2. **Степен на статична валидация** — кои аспекти на заявката и на модела могат да бъдат проверени преди изпълнение.

3. **Преносимост между backend-и** — възможно ли е логическият модел да бъде адаптиран към повече от един тип storage система.
4. **Сложност на toolchain-а** — изисква ли библиотеката външна кодогенерация, допълнителни preprocess стъпки или генератори на metadata.
5. **Прозрачност на грешките и на изпълнението** — доколко разработчикът може да проследи връзката между C++ кода, генерираната заявка и runtime поведението.
6. **Изпълнителен модел и асинхронна пригодност** — дали библиотеката предоставя единен и проверим модел за конкурентно или неблокиращо изпълнение, или оставя тази задача изцяло на конкретния драйвер и приложния код.

Тези критерии са пряко свързани с целите на дипломната работа. Ако предложената библиотека трябва да бъде едновременно expressive и формално проверима, тогава тя трябва да превъзхожда алтернативите не само по удобство, а по баланса между тези шест измерения.

2.3. Native клиентски библиотеки

SQLite [6], libpq и MySQLConnector/C са представителни за подхода, при който приложението конструира и подава заявките директно към драйвера. Това е най-универсалният и често най-бързият вариант, но поставя тежестта на correctness изцяло върху приложния код. Разработчикът трябва ръчно да поддържа синхрон между имената на полета, поредността на параметрите и реалната схема на базата данни. При промяна на backend-а се променя не само диалектът на заявката, но и целият API за връзка, подготвяне, bind и четене на резултати.

Този подход е особено проблематичен, когато един и същ домейн модел трябва да бъде свързан последователно с повече от един тип база данни. В такъв случай приложението започва да натрупва условни пътища, backend-специфични SQL низове и дублирана логика за hydration на резултати. След определен размер на системата това намалява не само удобството, но и проверимостта на коректността.

Силната страна на native API подхода е неговата честност. Той не обещава повече абстракция, отколкото реално предоставя. Именно затова той често служи като контролна точка, спрямо която могат да бъдат оценявани по-високите abstraction слоеве.

2.4. Асинхронност в съществуващите клиентски стекове

Съществена част от съвременния контекст е, че native клиентските библиотеки невинаги са само синхронни. libpq предоставя state machine за асинхронно изпращане и получаване на заявки [19], MySQL Connector/C предлага двойки от тип `_start/_cont` за неблокиращи операции [16], hiredis има callback-базиран async слой [17], а драйверът за Cassandra използва future/callback механизъм [3]. Тези възможности обаче са силно backend-специфични както по интерфейс, така и по execution semantics.

Оттук следва важен извод: наличието на native асинхронно API само по себе си не решава проблема за обща C++ абстракция. Във всеки отделен случай приложението трябва да знае дали работи със socket readiness цикъл, с callback регистрация, с future polling или с вътрешно управлявани драйверни нишки. При MongoDB акцентът остава върху pooling и безопасна многонитова употреба на libmongoc [14], а при libneo4j-client преобладава синхронният request/response модел [13]. Именно тази хетерогенност прави необходим междинен async слой, който да бъде достатъчно честен към backend различията и едновременно достатъчно единен за user-facing API.

2.5. Runtime query builder-и

Библиотеки като SOCI [8] и част от по-лекия fluent SQL tooling търсят баланс между четимост и контрол. Те предлагат по-удобен C++ интерфейс за изграждане на заявки, но в много случаи продължават да разчитат на runtime описване и последващо рендиране към SQL. В практиката това означава, че известна част от структурата на заявката е по-добре капсулирана, но backend-независимата смяна на модел за съхранение остава ограничена.

За настоящата разработка тези библиотеки са важни като отправна точка. Те показват, че fluent API и типизирани елементи са полезни, но не са достатъчни сами по себе си. Същинският въпрос е дали компилаторът може да валидира цялата логическа структура на заявката и дали същата абстракция може да бъде пренесена извън SQL света.

2.6. Типизирани SQL DSL решения

sqlpp11 [20] е особено интересен случай, защото стои между традиционния runtime query builder и по-строго статичния дизайн. Подобни библиотеки използват шаблони и типове, за да направят част от SQL структурата видима за компилатора. Това е съществена

стъпка напред спрямо чисто runtime builders, защото част от структурните грешки могат да бъдат хванати по-рано.

Същевременно обаче типизираният SQL DSL обикновено остава дълбоко свързан с релационния модел. Дори когато е много силен като C++ API, той най-често предполага свят, в който таблици, колони, joins и агрегати остават централната метафора. Това прави такива решения изключително ценни за SQL домейна, но по-трудно преносими към document, graph или key-value системи.

Именно тук се очертава една от основните разлики с настоящата разработка. Тя не цели просто compile-time структуриране на SQL. Тя цели compile-time логически слой, който може да бъде материализиран различно според backend семейството.

2.7. Класически ORM рамки и кодогенерация

ODB [2] е характерен пример за ORM рамка, която предлага богато картографиране между класове и релационни таблици, но използва генерация на код и външен tooling. Това улеснява потребителя на ниво употреба, но измества сложността към build процеса и към автоматично генерираните артефакти. Получава се силна зависимост между модела на данните, допълнителната toolchain стъпка и конкретните релационни backend-и.

В класическите ORM рамки често се наблюдава и друго напрежение: колкото по-високо е удобството на ниво application код, толкова по-непрозрачен става пътят до реалната заявка и до performance характеристиките на системата. Това не е задължително фатален проблем, но в академичния контекст на настоящата работа е важно ограничение, защото затруднява проследимостта между абстракцията и материализацията.

2.8. Граници на релационно центрираните абстракции

За приложения, които трябва да работят с документни, графови или key-value системи, класическият ORM модел показва естествени ограничения. Част от концепциите — като JOIN, външни ключове и строго таблична схема — нямат директен еквивалент във всички модели за съхранение. Поради това обобщението на ORM отвъд релационния свят не може да бъде постигнато само чрез генериране на SQL.

Това наблюдение е важно и за native API, и за SQL DSL решенията. Дори когато те са много силни технически, обикновено предполагат, че SQL е централният език на света на данните. Настоящата разработка приема по-различна отправна точка: логическият модел

трябва да бъде първичен, а релационната материализация — само една от възможните интерпретации.

Таблица 2. Ограничения на различните семейства решения спрямо multi-backend целта

Семейство решения	Какво прави добре	Къде се появява ограничението
Native API	максимален контрол и близост до драйвера	липса на общ логически модел и голямо дублиране при повече от един backend
Runtime builders	по-добра четимост и капсулиране на заявката	валидацията остава предимно runtime и често е SQL-центрична
Типизирани SQL DSL	по-силна статична структура за SQL заявки	логиката обикновено остава ограничена в рамките на релационната метафора
Класически ORM	удобно object mapping поведение за релационни системи	зависимост от кодогенерация и трудна преносимост към нерелационни модели

2.9. Позициониране на настоящата разработка

Предложената библиотека се позиционира като Compile-time Object-Relational Mapping решение, което комбинира силна статична типизация, липса на външна кодогенерация, connector-базиран модел за пренасяне на логическата абстракция към различни backend-и и coroutine-базиран асинхронен слой за изпълнение. В сравнение с native API подхода тя намалява дублирането на логиката и риска от късно открити грешки. В сравнение с runtime query builder решенията премества структурната валидация към компилатора. В сравнение с класическите ORM рамки тя избягва външни генератори и

поставя ясен акцент върху contract-а на конектора. В сравнение с backend-специфичните async API-та тя предлага единен корутинен модел, без да твърди, че всички драйвери имат еднакви неблокиращи възможности.

Ключовата разлика обаче е по-дълбока. Настоящата разработка не разглежда compile-time техниките като средство за “по-умен SQL builder”, а като начин за формализиране на логическия модел на достъпа до данни. Аналогично, тя не разглежда асинхронността като декоративен wrapper, а като отделен архитектурен проблем със собствени ограничения, lifecycle правила и backend-специфични execution path-ове. Точно това позволява следващите глави да преминат от корутинната теория към compile-time архитектурата, async слой, connector contract-а и реалните multi-backend сценарии.

2.10. Изводи от сравнителния анализ

Сравнението със съществуващите решения води до четири основни извода. Първо, native API подходът остава незаменим като baseline за контрол и performance, но не предлага удовлетворително ниво на логическа абстракция. Второ, runtime query builder-ите и типизираните SQL DSL решения подобряват работата със Structured Query Language (SQL), но не решават напълно проблема за backend-независим логически слой. Трето, класическите ORM рамки предлагат висока абстракция, но често за сметка на toolchain сложност и ограничена пригодност извън релационния свят. Четвърто, native асинхронните client API-та са ценни, но показват толкова различни execution модели, че сами по себе си не осигуряват единна потребителска абстракция.

Точно това позициониране оправдава по-детайлното разглеждане в следващата глава на отделен, но ключов въпрос: кои езикови и теоретични механизми правят възможна coroutine-базираната асинхронна архитектура, върху която по-късно стъпват compile-time ORM моделът и multi-backend изпълнителният слой.

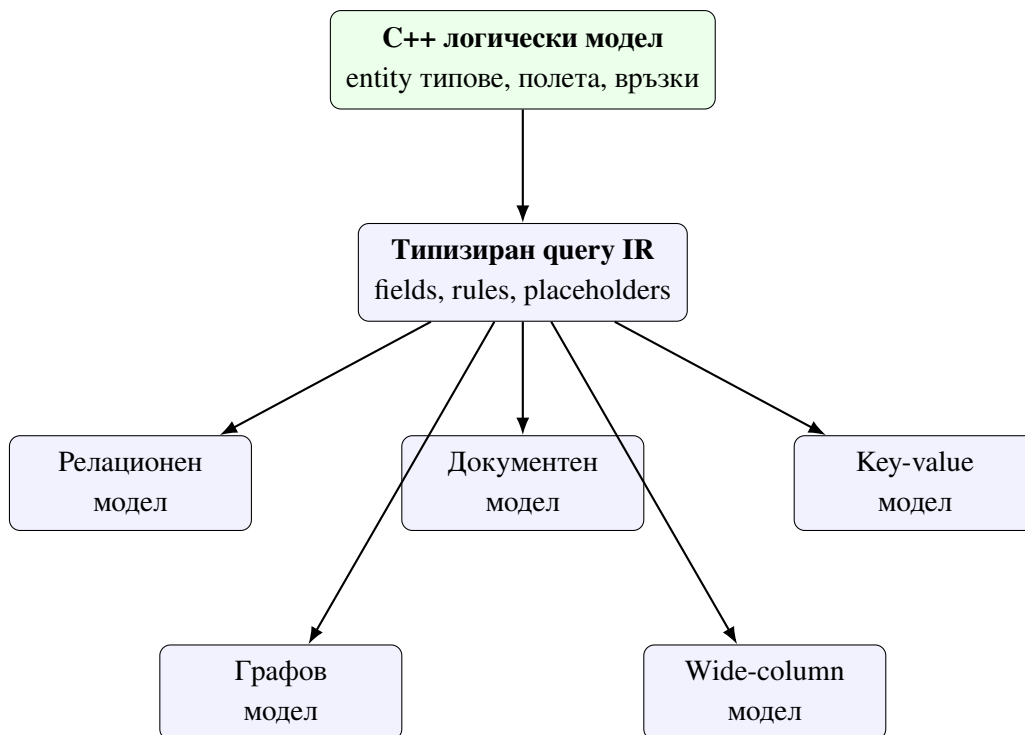
III. ТЕОРЕТИЧНИ ОСНОВИ НА COMPILE-TIME OBJECT-RELATIONAL MAPPING И МОДЕЛИТЕ ЗА СЪХРАНЕНИЕ

Теоретичната основа на настоящата разработка не се свежда до една езикова възможност или до един тип база данни. Библиотеката комбинира идеи от ORM теорията, статичното типизиране в C++, compile-time изграждането на междинни представяния и сравнителния анализ на различни модели за съхранение. За да бъде разбираема по-късната архитектура, е необходимо първо да се отдели логическият слой на описване на данните от физическия слой на тяхното backend-специфично материализиране.

В контекста на тази дипломна работа терминът Compile-time Object-Relational Mapping не трябва да се разбира стеснено само като SQL техника. По-точно той обозначава архитектурен подход, при който моделът на данните, формата на заявките и част от ограниченията върху операциите се описват чрез типовата система на езика, а не чрез runtime конфигурация. Това позволява една и съща логическа структура да бъде пренесена към релационни, документни, key-value, графови и wide-column системи, без backend-специфичните различия да се скриват или подменят.

3.1. Таксономия на problem domain-a

Преди разглеждане на отделните компоненти е полезно да се фиксира общата картина на problem domain-a. От една страна стои C++ моделът, в който приложението описва entity типове, полета, връзки и ограничения. От друга страна стоят различни семейства системи за съхранение, всяко със собствен модел на физическа организация, достъп и оптимизация. Между тях е разположен типизираният query IR, който служи като посредник между логиката на приложението и конкретния connector слой.



Фигура 2. Таксономия на логическия модел, query IR и backend семействата

Фигура 2 подчертава централното теоретично допускане на библиотеката: общото между backend-ите не е техният синтаксис, а логическият модел на данните и операциите. Следователно валидната абстракция трябва да бъде разположена по-високо от SQL, BSON, Redis команди, Cypher или CQL. Тя трябва да работи на равнището на полета, предикати, projection-и, връзки и допустими операции.

3.2. Compile-time Object-Relational Mapping като архитектурен подход

Класическото ORM свързва обектния модел на приложението с неговата персистентна схема [4]. В compile-time варианта това свързване се конструира не чрез runtime reflection и конфигурации, а чрез типове, шаблони и constexpr стойности. Практически това означава, че изборът на полета, допустимите операции и част от съвместимостта с backend-а се проверяват от компилатора. Грешки като обръщение към несъществуващо поле или използване на JOIN върху конектор без supports_joins престават да бъдат runtime изненади.

Съществената особеност е, че compile-time ORM не означава липса на runtime данни. Стойностите на параметрите се подават в момента на изпълнение, но структурата

на заявката е вече фиксирана и валидирана. Така се получава ясно разграничение между *логическа форма* и *runtime стойности*. Това разграничение е ключово за тезата, защото позволява едновременно статични гаранции и реална работа с външни бази данни.

От теоретична гледна точка този подход доближава ORM слоя до компилаторен pipeline. Приложният код описва семантична структура, compile-time компонентите я превръщат в междинно представяне, а едва най-долният слой материализира тази структура в backend-специфичен език или протокол. Затова по-късните архитектурни решения в библиотеката могат да се анализират през понятия като междинно представяне, статична проверка и backend materialization.

3.3. Статичен модел на данните в C++

В разглежданата библиотека домейн моделът се описва чрез C++ агрегати, чиито полета са от тип `property<T,Name>`. По този начин всяко поле носи едновременно стойностен тип и символно име на колоната или атрибута. Допълнително `table_name_trait<T>` позволява типът да бъде свързан с логически контейнер за съхранение — таблица, колекция, `key prefix`, `label` или друго backend-специфично понятие.

Тази организация има важно теоретично следствие: C++ типът става първичен носител на логическата схема. Отделните backend-и не налагат нов потребителски модел, а само интерпретират един и същ модел по различен начин. Именно затова по-късно може да се говори за еднакъв логически обект, който в релационен backend се превръща в таблица, а в документен — в документ с вложени или референтни полета.

Таблица 3. Съпоставка между C++ конструкциите и теоретичните ORM понятия

Конструкция	Теоретична роля	Практически ефект в библиотеката
<code>property<T,Name></code>	атрибут от логическата схема	носи тип на стойността и символно име за материализация
<code>relationship<...></code>	логическа връзка между обекти	отделя връзката от физическия механизъм за съхранение
<code>table_name_trait<T></code>	логически контейнер	служи като име на таблица, колекция, key prefix или label
<code>field<&T::m></code>	адресиране на атрибут в query пространство	свързва entity слоя с query IR-a
<code>connector_trait<DB></code>	backend материализация	определя как логическият модел се изпълнява физически

Следователно ORM слойът не започва от драйвер или от SQL рендериране. Той започва от статично описание на смисъла на данните. Това е и причината entity слойът да бъде фундаментален за цялата архитектура: ако този слой е неясен или непълен, няма как по-долу да се получи коректна и надграждаема материализация.

```

1  template <typename T, detail::string_literal Name>
2  struct property
3  {
4      using value_type = T;
5
6      [[nodiscard]] static constexpr std::string_view column_name() noexcept
7      {
8          return Name.view();
9      }
10
11     T value{};
12 };
13
14 enum class store_as { reference, embed };
15
16 template <store_as Strategy, typename T, detail::string_literal Name>
17 struct relationship
18 {
19     static constexpr store_as strategy = Strategy;
20     using related_type = T;
21
22     [[nodiscard]] static constexpr std::string_view field_name() noexcept

```

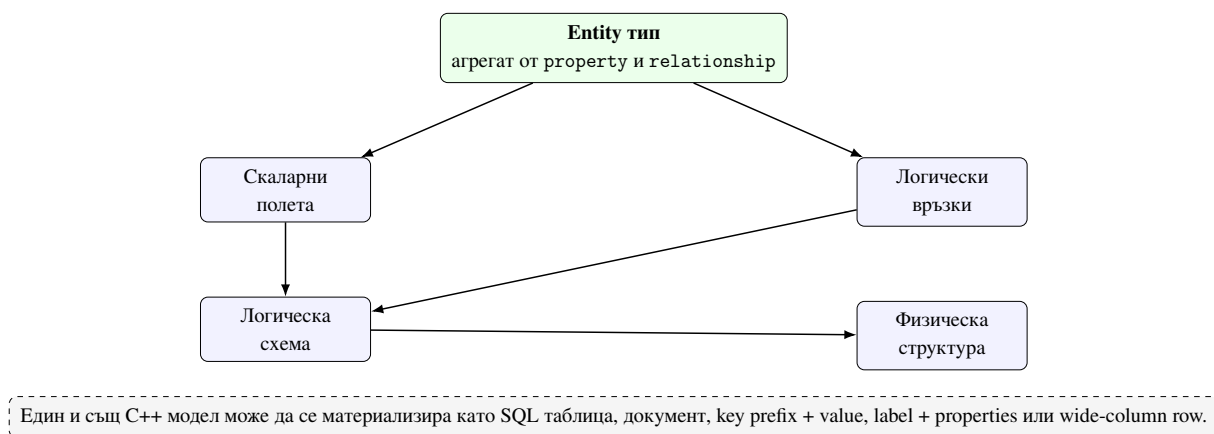
```

23 {
24     return Name.view();
25 }
26 };

```

Listing 1: Извадка от property и relationship в entity слоя

Listing 1 показва, че entity слойът е минималистичен по форма, но достатъчно богат по смисъл. property не е просто контейнер за стойност; той носи и име, а relationship не е просто member field; тя фиксира стратегията за материализация като част от типа.



Фигура 3. Преминаване от C++ entity описание към логическа схема и физическа материализация

3.4. Типизиран query IR

Вместо заявката да се описва директно като SQL или друго текстово представяне, библиотеката използва типизиран query IR. Неговите елементи са field, Rule, Placeholder и query обекти като select_query, insert_query, update_query и delete_query. Този модел дава две предимства. Първо, query структурата може да бъде анализирана на compile-time и да участва в static_assert проверки. Второ, един и същ IR може да бъде рендиран към различни backend езици и протоколи.

Теоретично IR-ът играе ролята на междинен език между домейн модела и конкретния слой за изпълнение. Това е аналогично на междинните представяния в компилаторите: потребителят работи с високо ниво на абстракция, а backend-ът получава специализирана форма, оптимизирана за собствения си модел. Именно поради това query IR-ът е логически по-близо до семантиката на операциите, отколкото до синтаксиса на конкретна заявка.

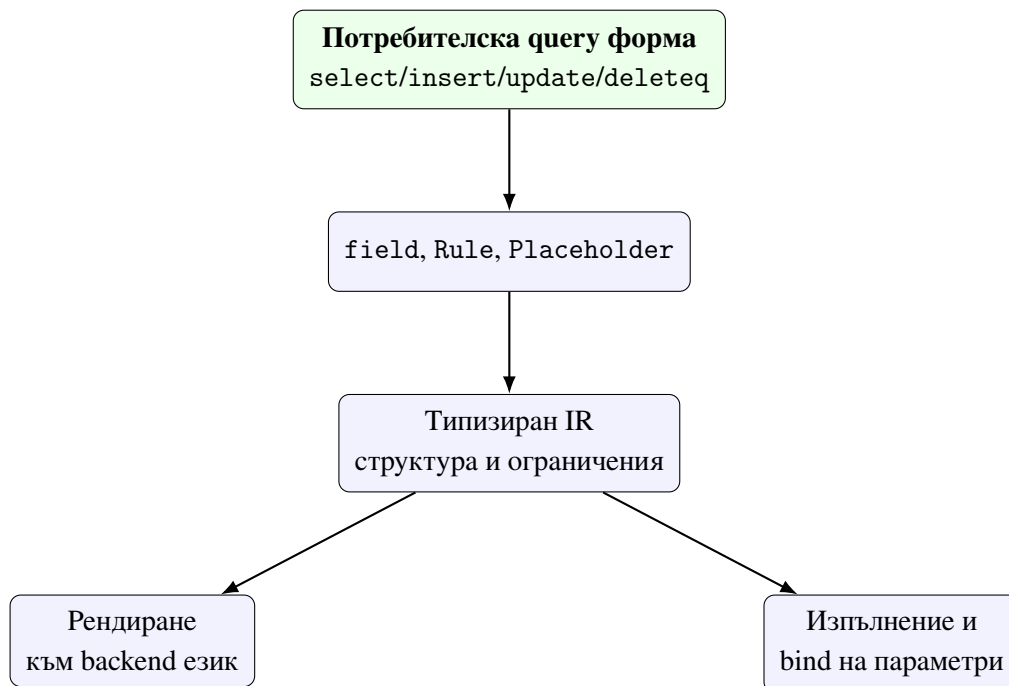

```

1 template <typename Response, typename Joins, typename Wheres,
2         typename Limits, typename Groups, typename Orders>
3 struct select_query : select_query_tag
4 {
5     using response_type = Response;
6     using row_type = projected_type<Response>;
7
8     template <typename RuleType>
9         requires is_rule<RuleType>
10    [[nodiscard]] constexpr auto where(RuleType r) const
11    {
12        using NewWheres = detail::append_type_t<Wheres, RuleType>;
13        return select_query<Response, Joins, NewWheres, Limits, Groups, Orders>(
14            response_, joins_,
15            detail::tuple_cat(wheres_, detail::orm_tuple<RuleType>(r)),
16            limits_, groups_, orders_);
17    }
18 };

```

Listing 2: Извадка от `select_query` и композицията на `where` клаузи

Listing 2 илюстрира важно теоретично свойство: `query builder`-ът не мутира текст, а изгражда нови типизирани обекти. Това означава, че последователността от операции `select(...).where(...).group_by(...)` може да бъде разглеждана като конструиране на нов тип, а не като поредица от странични ефекти върху низ.

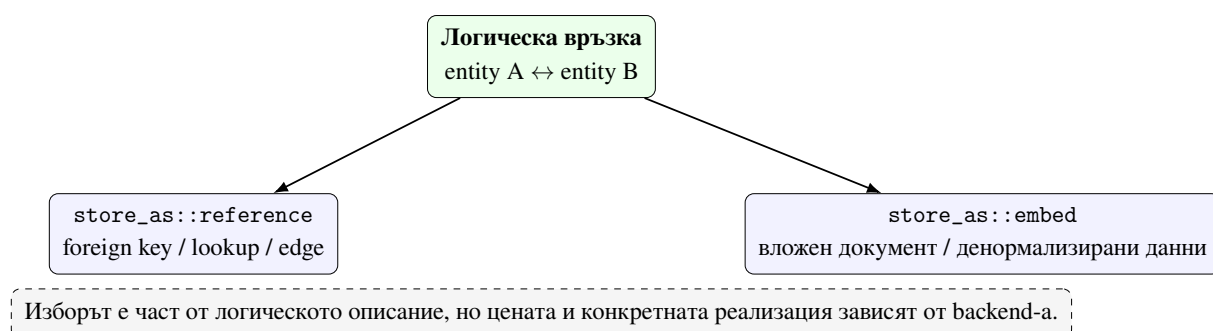


Фигура 4. Pipeline на типизирания `query IR` от приложния `API` до `backend` изпълнението

3.5. Теория на връзките и стратегии за съхранение

Връзките между обекти са един от най-трудните аспекти при обобщаване на ORM извън релационния свят. В релационен контекст една връзка често се материализира чрез външен ключ и JOIN. В документен контекст обаче същата логическа връзка може да се реализира като вграден документ, а в key-value контекст — чрез вторичен lookup или денормализиране.

Библиотеката абстрахира тези варианти чрез `relationship<Strategy,T,Name>` и `store_as::reference/store_as::embed`. `reference` обозначава логическа външна препратка, която backend-ът може да реализира чрез foreign key, lookup, отделен ключ или графова връзка. `embed` обозначава включване на свързаните данни в същата физическа структура. По този начин връзката се описва веднъж логически, а интерпретацията остава задача на конектора.



Фигура 5. Двете основни стратегии за материализация на връзки

Таблица 4. Сравнение между `reference` и `embed` стратегии

Стратегия	Логическа идея	Естествени backend-и	Основен компромис
reference	отделна идентичност на свързания обект	SQL, графови системи, key-based lookup	необходим е допълнителен механизъм за навигация или свързване
embed	физическо включване в родителската структура	документни системи, денормализирани записи	по-трудно независимо обновяване и споделяне на данни

3.6. Релационен модел

Релационният модел организира данните в таблици, редове и колони. Основните му предимства са ясна схема, декларативен език за заявки и добре дефинирани ограничения върху целостта. За ORM системите това е естествената среда: свойствата на C++ обекта лесно се съпоставят с колони, а връзките — с външни ключове. Операции като JOIN, GROUPBY и транзакции са част от нормалния езиков модел.

Именно релационният backend е естественият базов сценарий, спрямо който може да се оцени до каква степен една ORM библиотека запазва удобството си и извън SQL света. В настоящата разработка този базов сценарий по-късно се реализира основно чрез SQLiteDB, а преносимостта на IR-a се анализира допълнително чрез PostgreSQLDB и MySQLDB.

3.7. Документен модел

Документният модел организира данните в колекции от документи, чиито полета могат да бъдат вложени и не е задължително всички документи да имат еднаква физическа форма. Това го прави естествено съвместим с embed стратегията и по-гъвкав при денормализирани структури. От друга страна, твърде директното пренасяне на релационни абстракции върху документен backend може да бъде подвеждащо. Не всяка релационна операция има еквивалент със същата цена и семантика.

В контекста на настоящата работа документният модел е важен като тест за това дали query IR-ът описва логика, а не синтаксис на конкретен език. Ако един и същ IR може да се интерпретира като BSON-подобен филтър и projection, без потребителят да пише native document заявки, тогава действително е постигнат storage-agnostic слой на абстракция.

3.8. Key-value модел

Key-value системите организират достъпа около ключ и стойност. Те са особено ефективни, когато моделът на достъп е ясен и концентриран около lookup по идентификатор. Недостатъкът е, че общите predicate-базирани заявки и релационните операции са силно ограничени. Затова една ORM абстракция върху key-value backend не може да обещава същата изразителност като върху SQL система.

Този модел е полезен за разграничаване между *логически обект* и *допустим модел на достъп*. Обектът може да бъде един и същ, но не всяка заявка е смислена върху всяка физическа организация. В хранилището това по-късно се вижда ясно при RedisDB, където lookup логиката е естествена, а join-базираният стил е архитектурно неуместен.

3.9. Графов модел

Графовият модел представя данните като възли, ребра и свойства. Силата му е в изразяването на връзки като първокласни обекти, а не като производни от външни ключове. Това го прави подходящ за сценарии със сложни навигации и traversal операции. В същото време графовият модел налага собствен език на заявки и собствена интуиция за селекция и филтриране.

За библиотеката това е показателен сценарий, защото демонстрира, че connector_trait може да пренесе общ query IR към MATCH/RETURN/WHERE структура, без потребителят да пише Cypher директно. Ако това е възможно, значи общият слой действително е разположен над конкретния syntax layer.

3.10. Wide-column модел

Wide-column системите, каквато е Cassandra, се основават на разпределени таблици с partition и clustering ключове. Макар синтактично да напомнят релационни таблици, те имат различна теория на достъпа: правилната заявка е тази, която следва физическия ключов модел на разпределение. Следователно наличието на поле в C++ модела не означава автоматично, че може да бъде използвано свободно във WHERE клаузи.

Точно този тип ограничения оправдава capability и constraint подхода на библиотеката. Не е достатъчно query IR-ът да е типово коректен; той трябва да е и допустим спрямо модела на съхранение. По тази причина wide-column моделът е особено ценен за теоретичната защита на compile-time ограниченията.

Таблица 5. Съпоставка между основните модели за съхранение

Модел	Единица за съхранение	Типични връзки	Основно ограничение
Релационен	таблица/ред	foreign key, join	необходимост от съгласувана схема и силна типова дисциплина
Документен	документ	embed или reference	не всяка релационна операция е естествена или евтина
Key-value	ключ и стойност	lookup по ключ, hash полета	силно ограничени predicate заявки извън ключовия достъп
Графов	възел и ребро	traversal връзки	специфичен query език и графова семантика
Wide-column	partitioned row		заявките трябва да следват partition ключово-ориентираната зависимост

3.11. Тестова опора на теоретичните твърдения

Теоретичните твърдения в тази глава не са откъснати от кода. Entity слойт се валидира пряко в `tests/unit/test_property.cpp` и `tests/unit/test_relationship.cpp`, където се проверяват имената на колоните, разпознаването на entity типове, различаването между `store_as::reference` и `store_as::embed`, както и one-to-many inference логиката. Типизираният query IR е покрит в `tests/unit/test_query.cpp`, където се демонстрират field-based предикати, join форми, update composition и delete semantics. Допълнително `tests/unit/test_cpp26_reflection.cpp` показва, че compile-time описанието на полетата може да бъде надградено и чрез reflection-aware path, когато toolchain-ът го позволява.

Тази връзка между теория и тестова опора е съществена. Тя показва, че представените понятия не са свободни академични метафори, а реални архитектурни единици, които могат да бъдат проверявани в изходния код и в автоматизираните тестове.

3.12. Граници на унифицираната абстракция

След изложеното може да се направи важен извод. Унифицираната ORM абстракция не означава, че всички backend-и стават еднакви. Унифицирането е възможно на равнище логически модел, поле, връзка, query IR и базови операции. Отвъд тази граница започват различията: SQL JOIN няма универсален смисъл в Redis; embedding логиката няма идентична цена в класическа SQL таблица; graph traversal не се свежда до обикновен SELECT; partition-driven ограниченията в Cassandra не могат да бъдат маскирани като обикновени predicate правила.

Поради това архитектурно правилният подход не е да се скрият всички различия, а да се направят явни и формално контролирани. Именно тази роля изпълняват `connector_trait`, `capability tags` и `compile-time` ограниченията, разгледани в следващите глави. Точно там ще стане видимо как теоретичната рамка от настоящата глава се превръща в работеща архитектура на библиотеката.

IV. ТЕОРИЯ НА КОРУТИНИТЕ И АСИНХРОННОТО ИЗПЪЛНЕНИЕ

В C++

Корутините са езиков механизъм за описване на прекъсваеми изчисления, при които изпълнението може да бъде спряно и по-късно възобновено без загуба на локалното състояние. В стандарта C++20 корутините влизат в основния език, а не се реализират като външна библиотечна симулация [11, 10, 9]. Това е важно за настоящата работа, защото позволява асинхронният слой на библиотеката да се изгражда върху `compile-time` дефинирани правила за типове, `lifetime` и `control flow`, вместо върху `ad hoc callback` структура.

В тази глава корутините се разглеждат не като удобен синтаксис, а като изпълнителен модел, който стои между чисто синхронното блокиращо изпълнение и тежките нишки на операционната система. Целта не е да се покаже само как се използват `co_await` и `co_return`, а да се изведе защо компилаторното преобразуване към `state machine` прави този подход особено подходящ за библиотека, която и без това се основава на статични типове, `trait` специализации и `compile-time` валидиране.

Този теоретичен пласт е необходим и поради още една причина. В контекста на `Compile-time Object-Relational Mapping` корутините не са изолиран езиков експеримент, а основа за единен асинхронен интерфейс над `backend`-и с много различни `operational` характеристики. За да бъде такава архитектура научно защитима, трябва ясно да се разграничат езиковият механизъм, `runtime` инфраструктурата и конкретните драйверни `execution semantics`.

4.1. Процеси, нишки, задачи и корутини

Процесът е защитен адресен контекст на операционната система, нишката е `kernel`-планирана последователност от инструкции, а задачата е по-общо програмно понятие за единица работа. Корутината не е нито процес, нито нишка. Тя е кооперативна единица за изпълнение, чието спиране и подновяване се управлява от програмния модел и `generated runtime` структура, а не директно от `scheduler`-а на операционната система.

Това разграничение е съществено, защото библиотеката комбинира корутини и `ThreadPool`, без да приравнява двете абстракции. Нишката е скъпа `kernel` ресурсна единица, свързана с отделен `stack`, `scheduling` и синхронизационни примитиви. Корутината

е значително по-лека логическа единица, която пази само състоянието, необходимо за спиране и продължаване на конкретно изчисление. Следователно преминаването към coroutine-базиран модел не означава „повече нишки“, а по-структуриран начин за описване на това къде едно изчисление може да изчака външно събитие.

Таблица 6. Съпоставка между основните изпълнителни единици

Единица	Същност	Значение за библиотеката
Процес	Изолиран адресен контекст с ресурси на операционната система	Не е пряка единица на ORM архитектурата, но определя границата на runtime средата
Нишка	Kernel-планиран поток на изпълнение със собствен stack	Използва се от ThreadPool за offload на блокиращи операции
Корутина	Кооперативно прекъсваемо изчисление със запазено локално състояние	Осигурява формалния модел за Task<T> и за асинхронната композиция

Таблица 6 подчертава, че корутината е логическа, а не kernel единица. Именно това позволява в една и съща нишка да се редуват множество suspend/resume последователности, без всяка от тях да изисква отделен stack и отделна нишкова идентичност.

4.2. Синтаксис, awaitable protocol и езикови binding-и

Ключовите езикови оператори `co_await`, `co_return` и `co_yield` маркират, че дадена функция следва coroutine semantics. От гледна точка на архитектурата най-важен е `co_await`, защото именно той описва точката, в която текущото изчисление може да отстъпи изпълнението, докато външна операция — например база данни или scheduling към thread pool — не стане готова. Така синтаксисът на потребителя остава близък до

линейния код, докато изпълнителният модел става асинхронен.

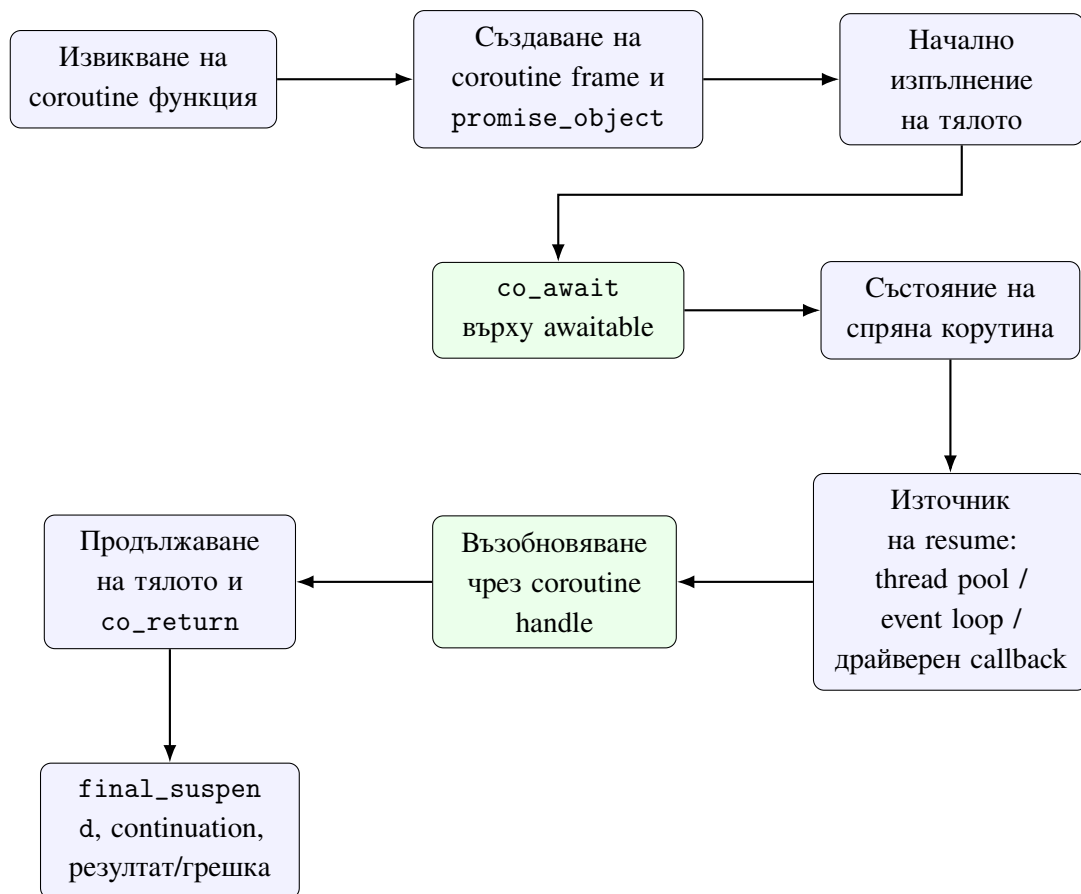
За текущата дипломна работа е важно, че тази езикова форма не е динамично „откривана“ в runtime, а е компилаторно вързана към return type-а на функцията и към promise_type, който std::coroutine_traits извежда за конкретния подпис [11, 10]. Това означава, че начинът, по който една корутината създава резултат, предава грешка или съхранява continuation, може да бъде формализиран още на ниво типова система.

На практика co_await работи чрез awaitable protocol. Обектът, който се await-ва, участва в тристъпкова схема: await_ready() определя дали спиране изобщо е необходимо, await_suspend() получава coroutine handle и решава как да организира възобновяването, а await_resume() връща резултата или повдига грешка. Точно тази тройка прави възможно различни източници на асинхронност — thread pool задачи, event loop readiness, callback-и от драйвер — да бъдат уеднаквени под общ co_await синтаксис.

4.3. Корутината като state machine

Компилаторът не „изпълнява магия“, а преобразува coroutine function в state machine с ясно определени suspend points, resume transitions и frame storage [10, 9]. Това преобразуване е от особено значение за настоящата теза, защото показва, че compile-time подходът на библиотеката не се изчерпва само с моделиране на заявки. Същият езиков инструментариум позволява и асинхронният control flow да бъде формализиран чрез типове и статични правила, вместо да се описва чрез неструктурирани callback вериги.

Концептуално една coroutine функция преминава през няколко ясно разграничени фази: създаване на frame и promise_object, начално изпълнение, достигане до suspend point, външно организирано възобновяване и финално завършване. Това е съществено, защото correctness вече не зависи само от „какво прави кодът“, а и от това кога е допустимо той да бъде продължен и кой носи отговорност за продължаването.



Фигура 6. Концептуален lifecycle на корутината като frame, suspend points и външно управлявано възобновяване

Фигура 6 показва защо корутинният модел е естествен за библиотека с ясно отделени слоеве. Логическият код на потребителя остава линеен, но реалното управление на изпълнението преминава през формални преходи, които могат да бъдат свързани с конкретни runtime механизми.

4.4. Coroutine frame, promise object, continuation и безопасност

При всяка корутина компилаторът създава coroutine frame, в който се съхраняват локалните променливи, текущото състояние на изпълнението и promise object-ът, чрез който корутината комуникира резултат, грешка или завършване. Това има директно отражение върху безопасността: lifetime на frame-а, ownership на резултата и мястото на continuation-а не са второстепенни детайли, а част от correctness contract-а на асинхронния слой.

Трябва да се подчертае, че coroutine frame-ът не е обикновен stack frame. Той може да преживее излизането от текущия стеков контекст и да бъде възобновен

по-късно от друга нишка или от event loop инфраструктура. Поради това проблеми като преждевременно разрушаване на frame-a, двойно възобновяване, неправилно предаване на exception състояние или загуба на continuation не са „ниско ниво implementation details“, а фундаментални correctness рискове.

Именно promise object-ът организира връщането на стойност, propagation-a на грешка и поведението при `initial_suspend` и `final_suspend`. От гледна точка на архитектурата това е важен пример за compile-time binding: видът на резултата и semantics на завършването не се определят от динамична registry схема, а са закодирани в coroutine return type-a. Това позволява `Task<T>` да бъде проектиран като формална абстракция, а не като неформален „future-подобен“ обект.

4.5. Корутини, thread pool и event loop инфраструктура

`ThreadPool` и корутините решават различни проблеми. `ThreadPool`-ът предоставя ограничен брой worker нишки и policy за планиране на работа, докато корутините описват как една логическа задача може да бъде спряна и по-късно възобновена. Комбинацията между двете позволява блокиращи backend-и да бъдат интегрирани по универсален начин чрез offload, без потребителят да се връща към callback-oriented програмиране.

От друга страна, при истински неблокиращи драйвери корутината може да бъде възобновявана не от worker нишка, а от event loop или callback bridge. Reactor-подобни среди наблюдават готовността на file descriptor-и чрез системни механизми като `kqueue` на macOS и BSD системите [1]. Следователно корутината сама по себе си не „изпълнява I/O“, а предоставя формален интерфейс, през който I/O readiness или driver callback може да се превърне в resume действие.

Точно тук става ясно защо корутините не трябва да бъдат смесвани с нишките. Нишката е носител на физическо изпълнение; корутината е носител на логическа последователност. Един и същ асинхронен модел може да използва thread pool в един backend сценарий и event loop в друг, без да променя потребителския синтаксис на `co_await`.

4.6. Значение за Compile-time Object-Relational Mapping библиотеката

За Compile-time Object-Relational Mapping този теоретичен анализ има пряко архитектурно следствие. Compile-time моделираният query IR и connector contract определят какво е позволено да се изпълни; coroutine-базираният слой определя как това позволено изчисление може да бъде изпълнено асинхронно. С други думи, типово коректната заявка и коректният execution path са различни, но взаимно зависими пластове.

Това разграничение е особено важно при multi-backend библиотека. PostgreSQL и MySQL предоставят native неблокиращи клиентски интерфейси, но с различни operational форми [19, 16]. hiredis използва callback-базиран async слой [17], а драйверът за Cassandra работи чрез future/callback механизъм [3]. При MongoDB фокусът остава върху безопасен pooling и многонитова употреба на libmongoc [14], докато libneo4j-client следва преобладаващо синхронен request/response модел [13]. Именно тази нееднородност оправдава асинхронна архитектура с две стратегии: универсален offload модел и native async интеграция за драйвери, които я позволяват.

Следователно корутините не правят автоматично всяка операция неблокираща и не отменят ограниченията на даден драйвер. Те дават формален модел за асинхронно описание на изчислението; реалният performance ефект зависи от това дали backend библиотеката има native non-blocking интерфейс, дали работата се offload-ва към thread pool или дали задачата остава синхронна. Именно затова следващите глави разглеждат отделно compile-time ORM архитектурата, асинхронния слой на рамката и execution semantics на конкретните backend-и.

V. COMPILE-TIME ORM АРХИТЕКТУРА

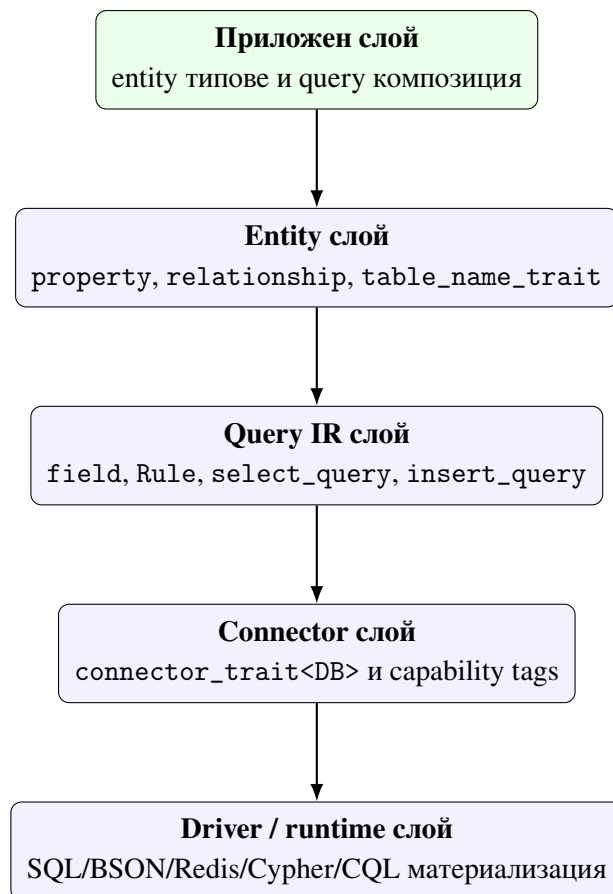
Compile-time ORM архитектурата на библиотеката е организирана около ясно разграничение между логическия модел на данните, междинното представяне на заявките и backend-специфичния слой за материализация. Именно този пласт носи първата основна научна ос на разработката: как компилаторът може да участва в моделирането, ограничаването и валидирането на достъпа до данни още преди runtime фазата. За да бъде постигнато това, архитектурата е изградена от няколко слоя, всеки със собствена отговорност и със строго определени интерфейси.

В теоретичната глава беше показано, че унифицирането е възможно на равнището на логическия модел и query IR-a, но не и чрез игнориране на различията между backend-ите. След корутинната теория от предходната глава тук фокусът съзнателно се връща към compile-time ядрото на системата: entity описанията, query IR-a, connector_trait<DB> и capability механизма. По този начин библиотеката се разглежда не просто като идея, а като организирана система от типове, contracts и формални граници на допустимите операции.

5.1. Архитектурни цели и ограничения

Архитектурата е проектирана при едновременно действие на няколко ограничения. Първо, user-facing API трябва да остане в рамките на идиоматичен C++, без въвеждане на отделен DSL и без runtime reflection механизъм. Второ, query структурата трябва да бъде представима като constexpr обект, така че значима част от валидирането да се извършва от компилатора. Трето, backend слойът трябва да бъде разширяем без наследяване от общ виртуален интерфейс, тъй като подобен интерфейс би скрил различията между backend-ите и би изместил проверките към runtime.

Четвъртото ограничение е свързано със самия multi-backend характер на системата. Библиотеката трябва да може да поддържа SQL и non-SQL конектори без фалшиво уеднаквяване. Това означава, че архитектурата трябва едновременно да допуска общ API и да изразява ограниченията на конкретен backend по формален и проверим начин. Оттук естествено следва изборът на trait-базиран connector слой и capability механизъм.



Фигура 7. Слоеста архитектура на библиотеката

Фигура 7 показва, че библиотеката следва последователна вертикална организация. Приложният код не комуникира директно с драйвера; той работи с C++ типове и query builder-и. Драйверът от своя страна не познава домейн семантиката, а само materialized формата, генерирана от connector слоя. Именно поради това архитектурата може да остане едновременно строга и надграждаема.

5.2. Слоеста организация и разделяне на отговорностите

Най-горният слой е приложният код, който дефинира entity типове, изгражда заявки чрез `select`, `insert`, `update` и `deleteq`, и ги подава на `db<DB>`. Следващият слой е логическият модел на библиотеката — entity описанията и query IR. Третият слой е connector abstraction-ът, където `connector_trait<DB>` материализира същата логика към backend-специфичен синтаксис и поведение. Най-долу стои конкретният драйвер, wire protocol или mock implementation.

Тази архитектура има важен практически ефект. Ако потребителят промени `SQLiteDB` с `MongoDB`, query IR остава логически същият, но конекторът го тълкува

като filter/projection изпълнение вместо SQL рендиране. Това не означава, че всеки IR е допустим върху всеки backend; означава, че *формата на абстракцията* е обща, а допустимостта се определя формално чрез capability механизма.

Таблица 7. Основни архитектурни слоеве и техните отговорности

Слой	Основни артефакти	Отговорност
Приложен	entity типове, query builder повиквания	описва домейн логика и формулира операции в идиоматичен C++
Логически	property, relationship, field, Rule	моделира данните и заявките независимо от конкретен backend
Connector	connector_trait<DB>, capability tags	проверява допустимостта и материализира IR-а към backend поведение
Runtime/driver	конкретни connection handle-и и драйверни API-та	изпълнява материализираната заявка и връща резултати

Таблица 7 показва, че библиотеката не използва “голям обект”, който да прави всичко. Вместо това различните решения са локализирани в отделни слоеве. Това е важно както за надграждаемостта, така и за академичния анализ, защото позволява отделните твърдения да бъдат проследени до конкретни типове и модули.

5.3. Entity слой като архитектурна основа

Entity слойът е изграден върху property<T,Name>, relationship<...> и table_name_trait<T>. Той има двойна роля. От една страна, представя C++ домейн модела. От друга страна, служи като логическо описание на схемата или структурата на персистентните данни. Това означава, че архитектурата не прави отделна runtime registry от метаданни. Вместо това типовата система съдържа достатъчно информация, за да бъдат извлечени имена на полета, типове, връзки и целеви контейнер за съхранение.

Особено важен е фактът, че entity слойът е независим от вида база данни. Нито property, нито relationship са SQL-специфични. Те описват логически атрибути и отношения. Така се създава основата за сравнимо поведение на релационни и нерелационни backend-и.

На архитектурно ниво именно това прави възможно библиотеката да твърди, че C++ моделът е първичният носител на логическата схема.

5.4. Query IR като централен архитектурен компонент

Query IR е централният архитектурен компонент. Той обединява field референции, предикати, placeholder-и и допълнителни клаузи като `order_by`, `group_by` и `limit`. Вместо да се работи с текст на заявка, библиотеката изгражда типизиран обект, чийто шаблонни параметри кодират логическата структура на операцията.

От архитектурна гледна точка това решение има няколко последствия. Първо, валидирането на структурата се извършва рано. Второ, query IR може да бъде анализиран от connector слоя без загуба на семантична информация. Трето, подготвените заявки могат да запазят не SQL низ, а самото междинно представяне, което е по-естествено за библиотека, ориентирана към C++ типовата система.

Четвърто, query IR-ът е единствената зона, в която се срещат entity описанието и по-късната backend материализация. В този смисъл той е архитектурният “шарнир” на цялата библиотека: достатъчно висок, за да остане storage-agnostic, и достатъчно конкретен, за да може да бъде изпълнен.

5.5. Конекторен слой и формалният contract

Формално connector слойът е реализиран чрез специализация на `connector_trait<DB>`. Този подход е аналогичен на стандартни trait шаблони като `std::iterator_traits`. Типът DB може да бъде прост wrapper над connection handle, а цялата ORM логика за него да остане в trait специализацията. По този начин библиотеката не изисква потребителят или авторът на конектора да наследява общ базов клас.

`connector_trait<DB>` определя поне три ключови аспекта: `wire_type`, `cursor_type` и `execute(...)`. По желание специализацията декларира и capability tags като `supports_joins`, `supports_transactions`, `supports_aggregation` или `supports_concurrent_execute`. Наличието или отсъствието на тези псевдо-маркери е достатъчно, за да може по-горният слой да наложи compile-time ограничения.

```
1 template <typename DB>
2 concept is_connector = requires {
3     typename connector_trait<DB>::template wire_type<int>::type;
```



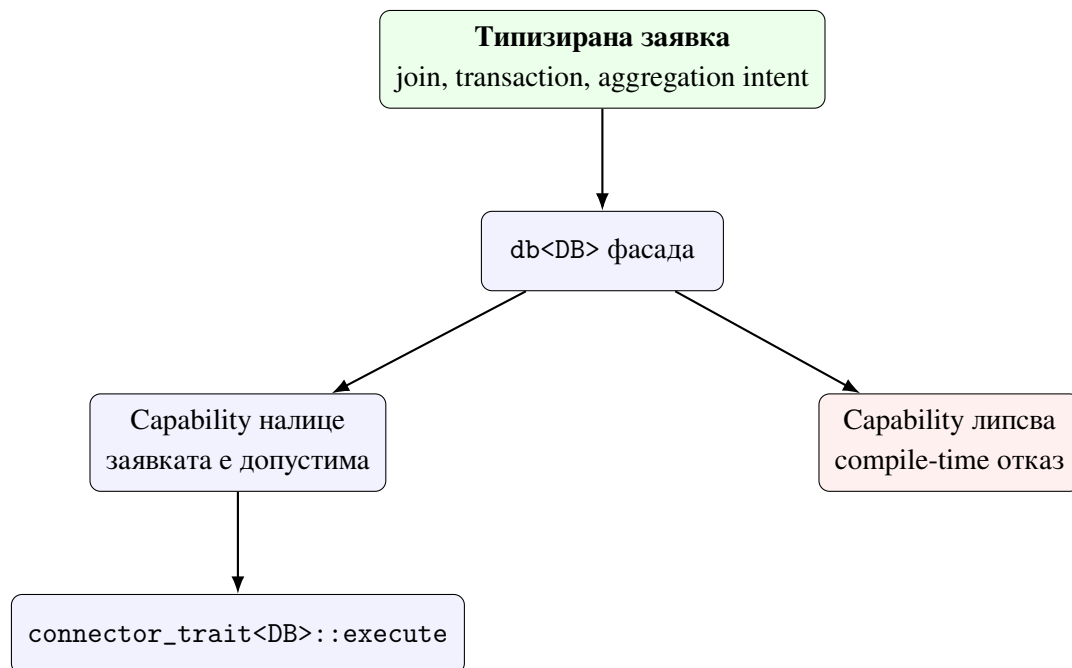
```

4     typename connector_trait<DB>::cursor_type;
5 };
6
7 template <typename DB, typename Cap>
8 inline constexpr bool has_capability =
9     detail::capability_check<DB, Cap>::value;

```

Listing 3: Извадка от формалния connector contract

Listing 3 показва, че contract-ът е структурен, а не наследствен. Това е решаващ архитектурен избор. Ако съществуваше виртуален интерфейс с runtime dispatch, част от типово проверимата информация щеше да бъде изгубена. Вместо това библиотеката предпочита compile-time структура, при която изискванията са видими и за компилатора, и за автора на конектора.



Фигура 8. Capability gating между query IR-а и connector слоя

Фигура 8 е важна, защото показва къде архитектурата взема решение за допустимост. Това не е драйверът и не е runtime логът. Решението се взема в публичния API слой, който вече има достъп до query IR-а и до trait информацията за конектора.

Таблица 8. Роля на основните capability tags в архитектурата

Capability	Архитектурен смисъл
supports_joins	разрешава join-базираните query форми за съответния backend
supports_transactions	позволява използване на transaction_guard и begin/commit/rollback hooks
supports_aggregation	обозначава поддръжка на aggregation операции в логическия query слой
supports_embedding	прави експлицитна native поддръжка на вградени структури
supports_upsert	маркира backend-и, при които upsert е част от естествения execution contract
supports_bulk_insert	обозначава наличието на по-ефективен път за масово записване

5.6. Потребителският фасаден слой db<DB>

Класът db<DB> е публичният фасаден слой. Той капсулира препратка към конкретния backend обект и предоставя уеднаквен API за изпълнение. На практика този слой има две роли. Първата е удобство за потребителя: библиотеката се използва чрез един и същ интерфейс независимо от backend-a. Втората е архитектурен филтър: точно тук се извършват compile-time проверки за capability и съвместимост.

Следователно db<DB> не е просто тънък wrapper над execute. Той е мястото, където библиотеката формализира границата между разрешено и неразрешено поведение за избрания backend. Точно този слой позволява съобщения за грешка от вида “конекторът не поддържа JOIN операции”, вместо грешката да се появи по-късно като неясно runtime поведение.

```

1 template <typename DB>
2     requires is_connector<DB>
3 class db
4 {
5 public:
6     template <typename Query>
7     auto operator<<(Query q)
8     {

```

```

9      if constexpr (is_select_query<Query>)
10     {
11         if constexpr (decltype(q.join_clauses())::size > 0)
12         {
13             static_assert(has_capability<DB, cap::supports_joins>,
14                           "orm::db: this connector does not support JOIN operations.");
15         }
16     }
17     return connector_trait<DB>::execute(*conn_, std::move(q));
18 }
19 };

```

Listing 4: Извадка от фасадния слой db<DB>

Listing 4 показва защо db<DB> е повече от удобна фасада. Той е формалното място, където query структурата и connector capability информацията се срещат. Именно тук compile-time архитектурата се превръща в реално ограничение върху API употребата.

5.7. Миграции като архитектурно разширение

Архитектурата включва и прототипен migration слой чрез migrate<DB>, live_schema, entity_meta и DDL операции като create_table_op, add_column_op, drop_column_op и alter_column_type_op. Този слой е особено важен от гледна точка на тезата, защото прави явна връзката между C++ модела и реалната схема на съхранение.

Миграционният слой не е универсално завършен за всеки backend, но архитектурно показва, че библиотеката не се ограничава само до query execution. При достатъчно богата специализация на connector_trait<DB> entity описанието може да участва и в еволюцията на схемата. Това променя и самия смисъл на ORM слоя: той вече не е само интерфейс към данните, а потенциален източник на schema-aware tooling.

5.8. Реализация на основните подсистеми

След като архитектурният модел е очертан, следващият въпрос е как той се материализира в конкретни C++ конструкции и как отделните подсистеми си взаимодействат в хранилището. Настоящият раздел възстановява липсващата инженерна перспектива: не само какви слоеве съществуват, а как точно се натрупват where клаузите, как query обектът се запазва за повторна употреба, как migrate<DB> превръща разлика между entity описанието и live_schema в DDL операции и как helper слоят налага правила за нишкова безопасност и транзакции.

Тук се фиксира и работната нотация, без да се връща отделна речникова глава. В настоящия раздел DB означава generic backend тип, db<DB> обозначава публичната фасада за изпълнение, connector_trait<DB> е формалният конекторен contract, field<...> е compile-time представяне на поле, Placeholder<T> е анонимен runtime параметър, а orm::ph<T, _N> е индексиран placeholder. Репозиторийният контекст на тази механика е концентриран главно в lib/include/ORM/entity/, lib/include/ORM/query/, lib/include/ORM/connector/, lib/include/ORM/db/migration/ и във верификационния слой под tests/unit/ и tests/integration/.

5.8.1. Мини казус с типовете User и Post

За да остане изложението непосредствено обвързано с реалния код, този раздел използва като опорна mini case study типовете User и Post от tests/integration/test_mockdb.cpp. Те са особено полезни, защото съчетават скаларни полета, явно именуване на логическите контейнери и join сценарий. Същият пример по-късно се използва и за placeholder binding, prepared query поведение и SQL rendering assertions.

```
1 struct Post
2 {
3     orm::property<int, "id">          id;
4     orm::property<int, "author_id">   author_id;
5     orm::property<std::u8string, "body"> body;
6 };
7
8 struct User
9 {
10     orm::property<int, "id">          id;
11     orm::property<std::u8string, "name"> name;
12     orm::property<double, "score">    score;
13 };
```

Listing 5: Entity типове от tests/integration/test_mockdb.cpp

Listing 5 показва не само как се описва entity моделът, но и как той непосредствено участва в тестовете. table_name_trait<User> и table_name_trait<Post> фиксират логическите контейнери "users" и "posts", а member pointer-и като &User::id и &Post::author_id по-късно участват в query builder-a. Това е пряка илюстрация на един от централните мотиви на дипломната работа: моделът, query формата и execution тестовете използват едни и същи compile-time артефакти.

5.8.2. Скалярни свойства и именуване на контейнерите

Реализацията на `property<T,Name>` свързва стойностен тип с `compile-time` име. Функцията `column_name()` дава еднозначен достъп до символното име на полето и се използва от конекторите при рендиране към SQL колони, `document fields`, `key-space` сегменти или други `backend`-специфични понятия. Конструкцията е минималистична, но достатъчна, защото не изисква външна `runtime` регистрация на метаданни.

Именуването на логическия контейнер за съхранение се реализира чрез `table_name_trait<T>`. Макар името да е наследено от ORM традицията, архитектурно то играе по-обща роля: в SQLite, PostgreSQL и MySQL означава таблица, в MongoDB — `collection`, в Redis — `key prefix`, а в Neo4j — `label`. Така библиотеката използва единен логически интерфейс за адресиране на физически различни структури.

Практическата последица е важна: авторът на конектора не извлича схема от външен `registry`, а от типовата система. Това намалява `boilerplate` кода и прави грешките в именуването по-лесни за локализиране както в компилационната фаза, така и в `unit` тестовете.

5.8.3. Field wrappers, правила и placeholder-и

Полето в `query` израз се представя чрез `field<&T::m>`. Това е `constexpr` wrapper над `member pointer`, който носи достатъчно информация за типа на контейнера, типа на стойността и името на колоната. Операторите върху `field` изграждат типизирани `Rule` обекти. Така изрази от вида `field<&User::id>==Placeholder<int>\{\}` не са низове, а `compile-time` описания на предикати.

`Placeholder` механизмът има две форми. `Placeholder<T>` моделира анонимни параметри, които се консумират позиционно от аргументите на `execute(). orm::ph<T,std::placeholders::_N>` моделира индексирани параметри, които позволяват една `runtime` стойност да се използва на повече от една позиция. Това е особено важно за `backend`-и като SQLite, PostgreSQL и MySQL, при които `placeholder` слотът е отделна част от `materialization` логиката.

Тестовете `IndexedPlaceholderRendersQuestionMarkN`, `IndexedPlaceholderReusedSameArgInSQL` и `TwoDistinctIndexedPlaceholdersRenderCorrectSlots` в `tests/integration/test_mockdb.cpp` показват, че тази подсистема не е теоретична добавка, а реално използван механизъм за по-точни `parameter binding` сценарии.

5.8.4. Изграждане на query обекти

Фабричните функции `select`, `insert`, `update` и `deleteq` създават query обекти, върху които последователно се наслагват `where`, `join`, `order_by`, `group_by` и `limit`. Същественото е, че всяка от тези операции не променя низ, а връща нов типизиран обект с разширено вътрешно състояние. В резултат query builder-ът остава едновременно fluent на вид и статичен по природа.

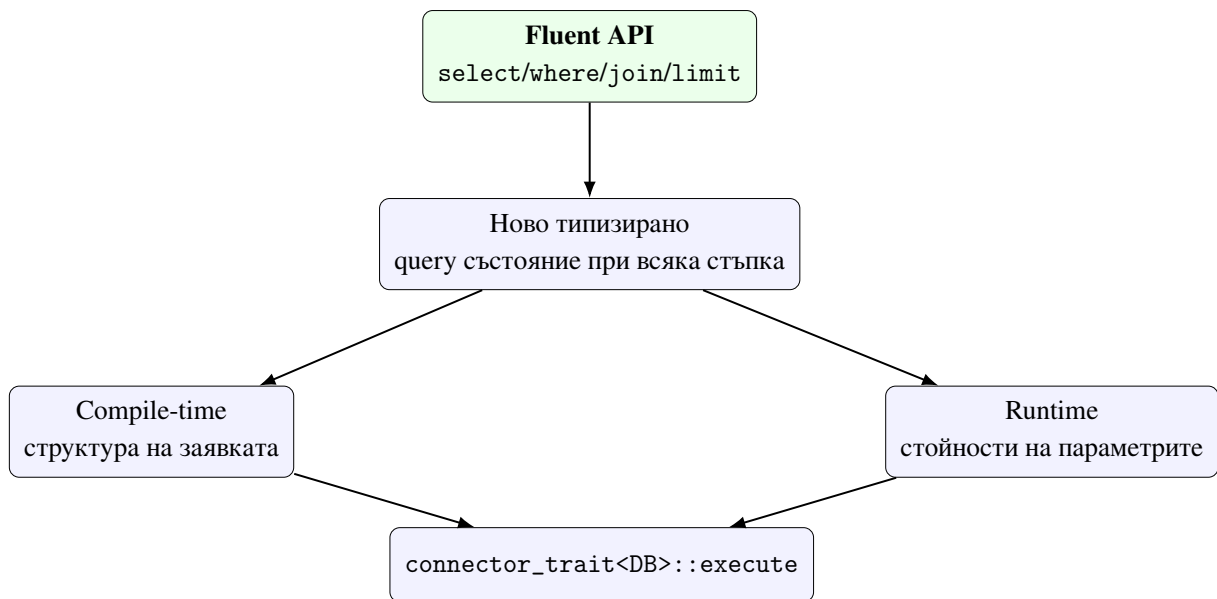
Това позволява в тестовете да се проверява не само крайният резултат от рендирането, а и самата форма на междинното представяне — например броят на `where` клаузите, типът на `response tuple`-а или коректността на `GROUPBY` и `ORDERBY` `composition`-а. В `tests/integration/test_mockdb.cpp` това се вижда в `SelectWithInnerJoin`, `SelectWithOrderByDesc`, `GroupByMultipleFields`, `SelectWithLimit` и множество други сценарии, които валидират различни фази на query `composition`.

Ключов имплементационен детайл е, че `builder` веригата е `constexpr`-съвместима. Вътрешният помощник `orm::detail::tuple_cat` използва разгъване на пакет чрез индексни последователности вместо зависимост от `runtime` конкатенация. Практическата последица е, че значима част от query формата може да бъде изградена и проверена още по време на компилация.

```
1 using namespace orm::literals;
2 static constexpr auto get_active =
3     orm::select(orm::field<&User::id>, orm::field<&User::name>)
4         .where(orm::field<&User::score> > 0.0)
5         .where(orm::field<&User::id> == orm::Placeholder<int>{})
6         .limit(10_per_page & 2_page);
```

Listing 6: Query обект деклариран като `compile-time` константа

Операторът `&` комбинира помощниците `_per_page` и `_page` в `Pagification` обект, чиято структура е известна по време на компилация. Хедърът `lib/include/ORM/query/limits.hpp` поддържа и симетрична форма чрез `*`, но използването на `&` в Listing 6 е по-четливо за заявчния стил на библиотеката. Така заявката може да бъде декларирана като `staticconstexpr` и да се използва многократно без реконструиране на `builder` формата при всяко извикване.



Фигура 9. Взаимодействие между query composition, compile-time структура и runtime параметри

Фигура 9 показва, че реализацията работи в два режима едновременно: структурната форма на заявката е фиксирана в типа, докато конкретните стойности се подават отделно към `execute(...)`. Именно това прави възможно едновременното наличие на compile-time проверки и runtime параметризация.

5.8.5. Изпълнение, резултати и достъп до полета

На етап изпълнение `db<DB>` предава query IR-a към `connector_trait<DB>::execute`. Резултатът се връща като `orm::result<Row, FieldTuple>`. В тази конструкция `Row` е материализираният tuple тип, а `FieldTuple` запазва projection metadata-то за избраните полета. Това решение позволява както итериране върху материализираните редове, така и достъп чрез `get_field<&T::m>` или позиционен достъп чрез `get<I>`. Реализацията така остава type-safe и след изпълнението на заявката.

Тестовите `SelectResultRowTypeIsTuple`, `SelectMultiFieldRowType`, `SelectGetByMemberPointer` и `SelectGetByIndex` показват, че result слой не е просто контейнер от анонимни редове, а структурирана абстракция с типово проследима проекция. Това е важно за тезата, защото демонстрира, че типовата дисциплина не приключва с query builder-a, а продължава и в result handling-a.

5.8.6. Prepared queries като повторно използваем execution артефакт

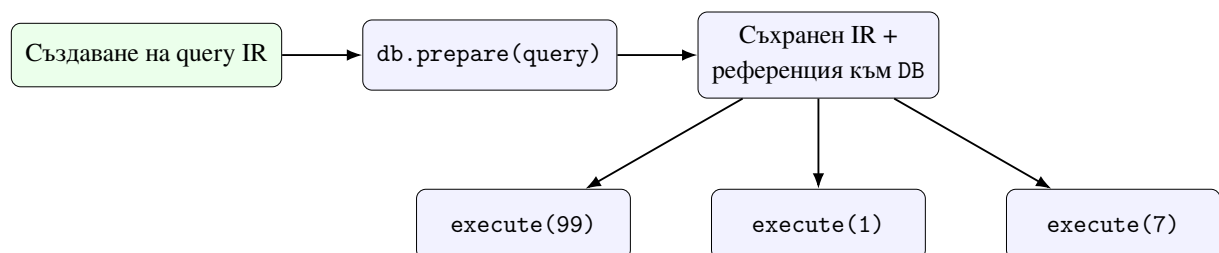
`prepared_query<DB, Query>` разширява изпълнителния модел. Вместо да кешира

SQL низ, той съхранява връзка към backend обекта и самия query IR. Това е естествено продължение на compile-time подхода: библиотеката третира query обекта като първичен артефакт, а не като временна стъпка по пътя към текстов резултат.

```
1 template <typename DB, typename Query>
2 class prepared_query
3 {
4 public:
5     prepared_query(DB& conn, Query q) : conn_(conn), query_(std::move(q)) {}
6
7     template <typename... Params>
8     auto execute(Params&&... params) const
9     {
10         return connector_trait<DB>::execute(
11             conn_, query_, std::forward<Params>(params)...);
12     }
13
14     [[nodiscard]] const Query& query() const noexcept { return query_; }
15 };
```

Listing 7: Извадка от prepared_query<DB,Query>

Listing 7 показва важен инженерен избор: повторната употреба е организирана около IR-a, а не около текстовата материализация. Това позволява една и съща логическа заявка да бъде изпълнявана многократно с различни параметри, без да се реконструира builder формата. Точно това поведение се валидира от PrepareReturnsExecutableObject, PrepareWithParamExecutesCorrectly, PrepareCanBeCalledMultipleTimesWithDifferentParams и PrepareExposesQueryIR.



Фигура 10. Lifecycle на prepared_query: един IR, множество изпълнения

5.8.7. Миграционен слой и DDL генериране

Реализацията на migrate<DB> сравнява описаната от entity типовете схема с live_schema. Когато има разлика, се създават DDL операции и се подават към connector_trait<DB>::ddl_for(...). Това е добър пример за clean separation на responsibilities: diff логиката е обща, а конкретният DDL синтаксис остава backend-специфичен.

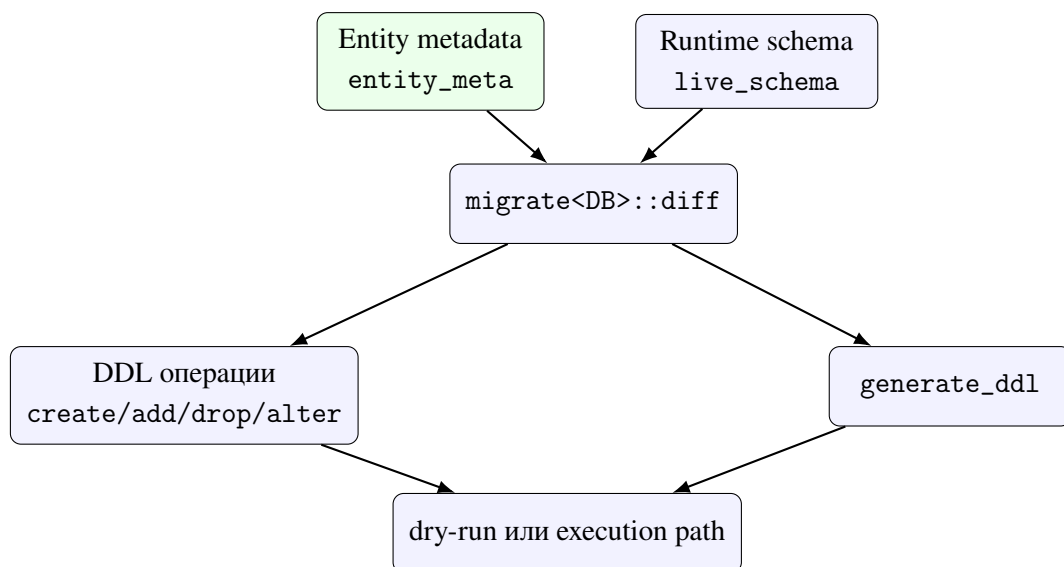

```

1 [[nodiscard]] std::vector<ddl_op> diff(
2     const live_schema& live,
3     std::span<const entity_meta> entities) const
4 {
5     std::vector<ddl_op> ops;
6
7     for (const auto& entity : entities)
8     {
9         auto live_it = live.find(entity.table_name);
10        if (live_it == live.end())
11        {
12            ops.push_back(create_table_op{entity.table_name, entity.columns});
13            continue;
14        }
15
16        for (const auto& [col, type] : entity.columns)
17        {
18            if (live_it->second.find(col) == live_it->second.end())
19                ops.push_back(add_column_op{entity.table_name, col, type});
20        }
21    }
22    return ops;
23 }

```

Listing 8: Ядро на migration diff pipeline-a

Тук ясно се вижда разликата между *минимален работещ конектор* и *пълен connector contract*. Един backend може да изпълнява заявки, без да поддържа DDL генериране. Но ако трябва да участва в schema-aware tooling, неговият contract трябва да бъде разширен с `ddl_for(...)` hooks и последователна migration верификация.



Фигура 11. Поток на schema migration подсистемата

Тестовите в `tests/unit/test_schema_migration.cpp` покриват създаване на таблица, добавяне и отпадане на колони, dry-run кодове на завършване, генериране на DDL и state-machine поведението на `run()`. Това е силно доказателство, че migration слойът не е само концепция, а реално проследим pipeline.

5.8.8. Нишкова безопасност и transaction helpers

В библиотеката съществува отделен слой за `connection_pool<DB,N>`, `thread_local_db<DB>` и `transaction_guard<DB>`. Тези компоненти показват, че реализацията не спира до еднократно изпълнение на заявка, а разглежда и operational сценарии с повторна употреба на връзки, изолиране по нишки и rollback дисциплина. Важното архитектурно решение е, че достъпът до тези механизми също е capability-базиран. Например `connection_pool` изисква `supports_concurrent_execute`, а `transaction_guard` — `supports_transactions`.

```
1  template <typename DB, std::size_t N>
2  class connection_pool
3  {
4      static_assert(
5          requires { typename connector_trait<DB>::supports_concurrent_execute; },
6          "connection_pool requires "
7          "connector_trait<DB>::supports_concurrent_execute. "
8          "Add 'using supports_concurrent_execute = void;' "
9          "to connector_trait<DB>.");
10
11  public:
12      [[nodiscard]] connection_guard<DB> acquire();
13  };
14
15  template <typename DB>
16  class transaction_guard
17  {
18      static_assert(
19          requires { typename connector_trait<DB>::supports_transactions; },
20          "transaction_guard requires "
21          "connector_trait<DB>::supports_transactions. "
22          "Add 'using supports_transactions = void;' "
23          "to connector_trait<DB>.");
24  };
```

Listing 9: Извадка от thread-safety helper слой

Така compile-time contract-ът не се ограничава до query формата, а достига и до lifecycle политиките на конектора. Това е важно за аргумента на тезата, че библиотеката

не е просто builder за заявки, а инфраструктура за систематична и безопасна работа с backend ресурси.

Таблица 9. Основни подсистеми и тяхната тестова опора

Подсистема	Основни файлове	Тестова опора
Entity моделиране	ORM/entity/property.hpp, ORM/entity/relationship.hpp	tests/unit/test_property.cpp, tests/unit/test_relationship.cpp
Query composition	ORM/query/select.hpp и свързаните query headers	tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp
Prepared queries	ORM/connector/prepared_query.hpp	Prepare* сценариите в tests/integration/test_mockdb.cpp
Schema migration	ORM/db/migration/migration.hpp	tests/unit/test_schema_migration.cpp
Thread safety	ORM/db/connectors/ThreadSafety/thread_safety.hpp	tests/unit/test_thread_safety.cpp

5.8.9. Извод за реализацията

Основните подсистеми имат пряка опора в кода и тестовете. tests/unit/test_property.cpp и tests/unit/test_relationship.cpp валидират entity описанията. tests/unit/test_query.cpp доказва, че query builder-ът действително конструира правилни compile-time структури. tests/integration/test_mockdb.cpp и tests/integration/test_sqlite.cpp валидират изпълнението, prepared query сценариите и result handling-a. tests/unit/test_schema_migration.cpp покрива diff логиката на миграциите, а tests/unit/test_thread_safety.cpp проверява lifecycle слоя за конкурентен достъп.

Следователно реализацията на основните подсистеми не е само проектно намерение, а архитектурно решение, подкрепено от систематична верификация. Това е и най-важният извод от настоящия раздел: compile-time ORM архитектурата в хранилището не остава на равнище абстракция, а се материализира в работещи подсистеми с ясно

разграничени отговорности и проследима репозиторийна опора.

5.9. Архитектурни инварианти

На базата на разгледаните слоеве могат да се формулират няколко инварианта. Първо, приложният код никога не бива да зависи пряко от драйверен API. Второ, query логиката се формулира в типизиран IR, а не в backend-native низове. Трето, разширяването към нов backend се извършва чрез trait специализация и capability декларации, а не чрез промяна на самия query API. Четвърто, ако една операция е недопустима за даден backend, това трябва да стане видимо възможно най-рано — за предпочитане при компилация.

Тези инварианти правят архитектурата едновременно инженерно подредена и академично защитима. Те позволяват всяко следващо разширение да бъде оценявано не само по това дали “работи”, а и по това дали запазва вече формулираните слоеве и граници.

5.10. Архитектурен извод

Обобщено, архитектурата на библиотеката разделя проблема на три равнища: логически модел, типизиран query IR и backend materialization. Това разделение позволява едновременно строга типова дисциплина, формализирана разширяемост и контролирано поведение върху различни модели за съхранение. В следващата глава фокусът се премества към втората архитектурна ос — асинхронното изпълнение — където става ясно как същото compile-time ядро се свързва с coroutine lifecycle, thread-pool offload и native non-blocking интеграция.

VI. АСИНХРОННА АРХИТЕКТУРА НА ИЗПЪЛНЕНИЕТО

След като `compile-time` архитектурният модел е дефиниран, следващият въпрос е как библиотеката реализира асинхронното изпълнение като втора основна архитектурна ос. Настоящата глава разглежда основните `building blocks` на `async` слоя и ги проследява от `Task<T>` и `co_await`-базираната композиция до `ThreadPool`, `async_db<DB>`, `IoContext`, `coroutine-aware connection pooling` и интеграцията с `backend`-специфични `execution path`-ове.

Тук фокусът вече не е върху `query IR`-а като `compile-time` структура, а върху реалната механика на `suspend/resume` изпълнението. Как синхронният `connector API` се повдига до асинхронен интерфейс? Къде `offload` към `thread pool` е правилната стратегия и къде `native non-blocking API` е по-точният модел? Как `async` слойът гарантира коректна употреба на връзките и транзакциите без връщане към `callback-driven` интерфейс? Отговорът на тези въпроси показва доколко библиотеката е завършена не само като типова, но и като изпълнителна система.

6.1. Архитектурна позиция на `async` слоя

Асинхронният слой е разположен над вече съществуващия синхронен модел `db<DB>` и `connector_trait<DB>`. Това е важен архитектурен избор. Вместо да дублира `query builder`-а или да въвежда втори набор от `query` типове, `async` подсистемата приема `compile-time` конструирания `query IR` като готов вход и се занимава единствено с това *как* да бъде изпълнен. По този начин първата архитектурна ос — типово описание на данните и операциите — остава отделена от втората ос — управление на асинхронния `lifecycle`.

От тази гледна точка `async` архитектурата има три основни отговорности. Първо, да осигури `coroutine` носител на `execution state` и `continuation`. Второ, да даде универсален механизъм за повдигане на `blocking connector`-и към `Task`-базиран интерфейс. Трето, да позволи `native non-blocking backends` да участват в същия потребителски синтаксис, без библиотеката да симулира фалшива еднородност там, където драйверите реално са различни.

6.2. Task<T> като носител на coroutine lifecycle

Основната потребителска абстракция е Task<T>. Нейната роля не се изчерпва с това да „върне бъдещ резултат“. Тя е контейнер за coroutine frame-a, promise object-a, continuation-a и политиката за завършване. Реализацията използва initial_suspend() и final_suspend() така, че корутината да бъде lazy по начало и да предава управлението обратно към continuation-a при завършване. Следователно Task<T> е едновременно execution handle и formal contract за това как резултат, exception и завършване се предават между coroutine контексти.

От инженерна гледна точка са важни три свойства. Първо, operator co_await() свързва текущата continuation към awaited задачата и позволява естествено вложено co_await композиране. Второ, sync_wait() осигурява мост към не-корутинен код, което е необходимо както за unit тестовете, така и за boundary сценарии, в които приложението още не работи изцяло coroutine-first. Трето, detach() и start_detached() позволяват controlled fire-and-forget поведение, при което coroutine frame-ът се самоунищожава след final_suspend().

Тази конструкция не е декоративна. Тя определя кога async операция е lazy, кога започва да се изпълнява, кой носи отговорност за lifetime-a на frame-a и как се propagated-ват exceptions. Именно поради това Task<T> стои в центъра на всички останали async компоненти: run_on_pool(), async_db<DB>, IoContext awaitable-ите и transaction helper-ите работят през един и същ coroutine договор.

6.3. ThreadPool и run_on_pool(): универсалният execution path

Универсалната async стратегия е thread-pool offload. ThreadPool поддържа bounded набор от worker нишки и опашка от задачи. Върху него стъпва run_on_pool(), който приема callable, публикува го към опашката, спира текущата корутина и я възобновява, когато callable-ът завърши. По този начин synchronous connector API може да бъде използван през co_await, без потребителският код да се връща към manual promise/future plumbing.

Критичният детайл е, че run_on_pool() не е просто helper за „пусни нещо на друга нишка“. Неговият awaiter пази continuation, евентуалния exception и резултата от callable-a. След изпълнение върху worker нишката continuation-ът се resume-ва, а await

`_resume()` връща резултат или повдига грешка. Така `offload path`-ът остава подчинен на същите `coroutine` правила като `native async path`-а.

Този универсален модел е особено важен за `backend`-и, които нямат естествен `native non-blocking` клиентски интерфейс или при които библиотеката съзнателно не претендира такъв. В тези случаи честният архитектурен избор е `blocking` операцията да бъде изолирана в `ThreadPool`, а потребителят да получи единен `Task`-базиран интерфейс. Точно това поведение се валидира от `RunOnPoolTest.*` и от `AsyncDbTest.AsyncSelectRunsOnPoolThread`, където се доказва, че реалното изпълнение се премества извън главната нишка.

6.4. `async_db<DB>` и `capability-gated dispatch`

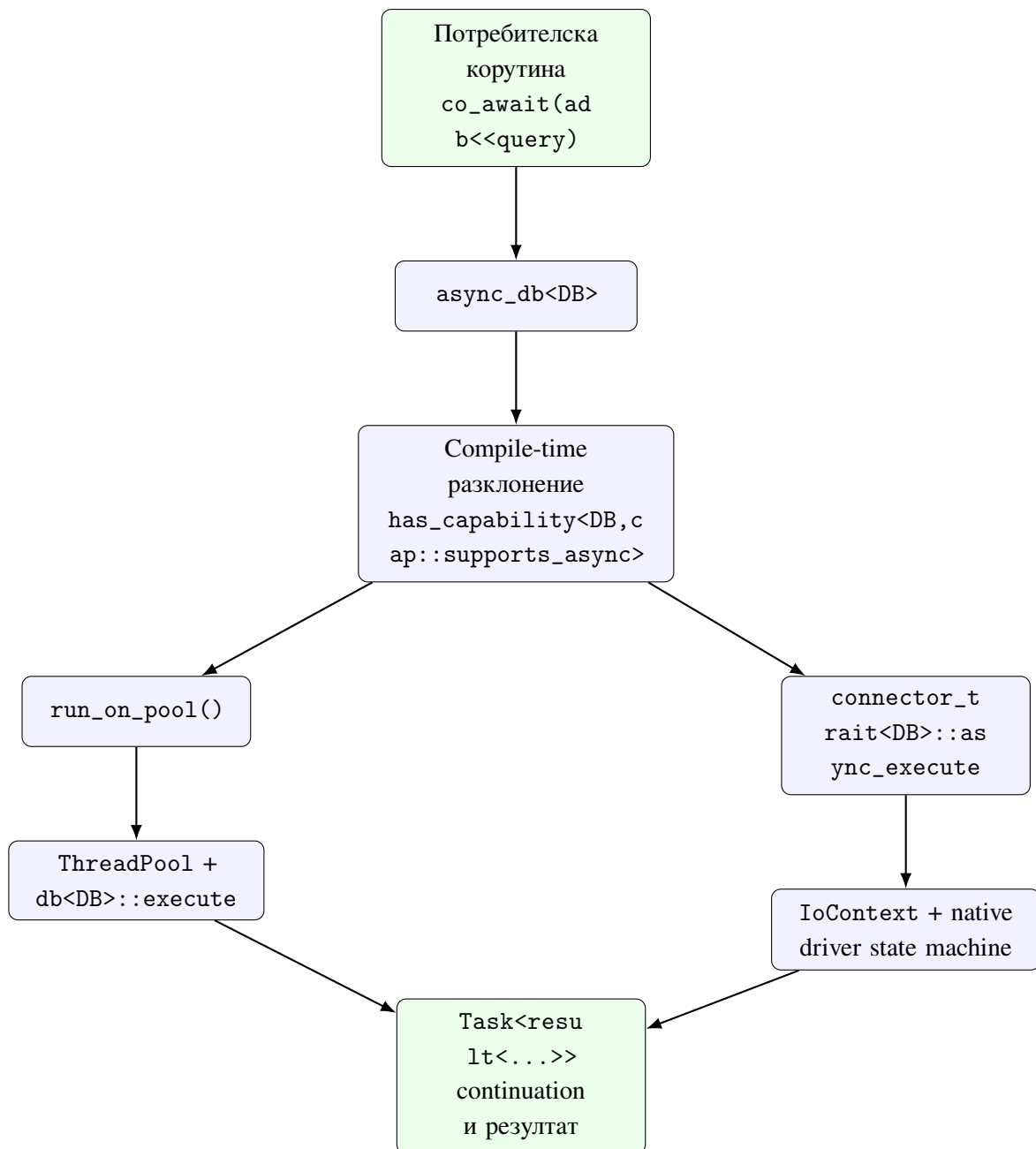
Най-важната външна `async` фасада е `async_db<DB>`. Тя обгръща `db<DB>` и запазва `familiar query syntax`-а, но връща `Task<result<...>>`. Архитектурното значение на тази обвивка е, че тя не решава динамично какво да направи в `runtime`, а използва `compile-time capability gating`. При `operator<<` има `ifconstexpr` разклонение върху `has_capability<DB, cap::supports_async>`. Ако `connector`-ът декларира `native async` възможност, се извиква `connector_trait<DB>::async_execute()`. В противен случай синхронният `execute()` път се обгръща в `run_on_pool()`.

```
1 template <typename Query>
2 [[nodiscard]] auto operator<<(Query q)
3     -> Task<decltype(std::declval<db<DB>>()) << std::declval<Query>())>
4 {
5     using ResultType = decltype(std::declval<db<DB>>()) << std::declval<Query>());
6
7     if constexpr (has_capability<DB, cap::supports_async>)
8     {
9         co_return co_await connector_trait<DB>::async_execute(
10             db_.connection(), std::move(q));
11     }
12     else
13     {
14         co_return co_await run_on_pool(
15             *pool_, [this, q = std::move(q)]() mutable -> ResultType {
16                 return db_ << std::move(q);
17             });
18     }
19 }
```

Listing 10: Централното `capability-gated` разклонение в `codeasync_db<DB>`

Listing 10 е централен за цялата async архитектура. Той показва, че библиотеката не използва runtime registry или string-based dispatch, а compile-time проверка върху connector capability-та. По този начин потребителският API остава еднороден, докато вътрешният execution path остава верен на реалните възможности на дадения backend.

Помощният метод `async_execute(Query, Params...)` изпълнява сходна bridging роля за imperative сценарии с явни параметри, а `sync_handle()` позволява controlled escape към синхронния интерфейс, когато такава граница е необходима. Следователно а `sync_db<DB>` не е паралелна библиотека, а адаптационен слой между compile-time ORM ядрото и coroutine execution модела.



Фигура 12. Двата execution пътя на `async_db<DB>`: универсален offload и native async интеграция

Фигура 12 показва най-важното архитектурно решение в `async` слоя. Външният интерфейс остава единен, но вътрешният execution path е capability-gated. Това е причината библиотеката едновременно да предлага удобен coroutine API и да избягва фалшивото твърдение, че всички драйвери са естествено неблокиращи.

6.5. IoContext и native non-blocking integration

За драйвери с истински native async интерфейс библиотеката не се ограничава до thread-pool offload. Тук влиза в действие IoContext — абстракция за event loop, която предоставя `run()`, `run_one()`, `stop()`, `post()`, `schedule()` и най-важното `watch_readable(fd)` и `watch_writable(fd)`. Тези awaitable-и не извършват входно-изходна операция сами по себе си. Те само спират корутината и я възобновяват, когато file descriptor-ът стане готов за четене или запис.

Точно това разделение между readiness и самото I/O действие е решаващо. IoContext не се опитва да „скрие“ драйвера, а предоставя coroutine-friendly reactor интерфейс, върху който native клиентската библиотека може да стъпи. В реализацията на AsyncPostgreSQLDB това се вижда особено ясно. Асинхронното свързване използва `PQconnectPoll()`, а корутината последователно await-ва `watch_writable()` или `watch_readable()` според състоянието на libpq state machine-a. При изпълнение на заявка `PQsendQueryParams()`, `PQflush()`, `PQisBusy()` и `PQconsumeInput()` се комбинират със същия IoContext механизъм.

Следователно native async пътят не е „по-бърз thread pool“, а qualitatively различен execution модел. Worker нишката не блокира в очакване на мрежова готовност; корутината се suspend-ва, а възобновяването се управлява от event loop наблюдение. Това е precisely архитектурният модел, който следващата глава развива по backend-и: PostgreSQL, MySQL, Redis и Cassandra могат да предложат native async интеграция с различни operational форми, докато други драйвери остават честно описани като blocking и следователно се обслужват от универсалния offload path.

6.6. Coroutine-aware connection pooling и transaction helpers

Async архитектурата не приключва с единичното изпълнение на заявка. Реалното приложение трябва да управлява връзки и транзакции при конкурентен достъп. Именно затова библиотеката предоставя `async_connection_pool<DB,N>`, `async_connection_guard<DB>`, `async_transaction_guard<DB>` и factory coroutine-a `async_begin_transaction()`.

`async_connection_pool<DB,N>` е coroutine-aware продължение на синхронния pooling слой. Неговият `acquire()` метод връща `Task<async_connection_guard<DB>`

>. Вътрешно той използва `run_on_pool()` и `condition variable`, за да изчака `connection slot` със състояние `Idle`, без `calling coroutine`-ата да бъде изложена на `blocking interface`. Полученият `guard` следва идиомата `Resource Acquisition Is Initialization (RAII)`: при разрушаване той връща състоянието на връзката към `Idle` и уведомява чакащите.

От своя страна `async_connection_guard<DB>::get()` не връща сурова връзка, а нов `async_db<DB> handle`. Това е важен архитектурен детайл. Дори след придобиване на конкретна `connection resource` потребителят остава в същия `coroutine execution` модел и не пада обратно към ниско ниво `API`.

Сходна логика се вижда и при транзакциите. `async_begin_transaction()` first offload-ва `BEGIN` към `pool-a`, а след това връща `async_transaction_guard<DB>`. Методите `commit()` и `rollback()` са `Task<void>` операции, а деструкторът на `guard-a` изпълнява защитен `ROLLBACK`, ако транзакцията не е финализирана изрично. Този дизайн е едновременно удобен и консервативен: `success path`-ът е `coroutine-friendly`, а `cleanup path`-ът остава `deterministic`.

Точно тези свойства се валидират от `AsyncConnectionPoolTest.ConcurrentAcquires`, `AsyncTransactionTest.BeginAndCommit`, `AsyncTransactionTest.RollbackOnDestruction` и `AsyncTransactionTest.ExplicitRollback`. Следователно `pooling` и `transaction helper`-ите не са периферни удобства, а важна част от цялостната `execution` архитектура.

Таблица 10. Основни async подсистеми и тяхната кодова и тестова опора

Подсистема	Архитектурна роля	Основни файлове и тестове
Task<T>	coroutine lifecycle, continuation, sync_wait(), detach semantics	lib/include/ORM/async/task.hpp, tests/unit/test_task.cpp
ThreadPool и run_on_pool()	универсален offload path за blocking операции	lib/include/ORM/async/thread_pool.hpp, tests/unit/test_thread_pool.cpp
async_db<DB>	capability-gated async dispatch над db<DB>	lib/include/ORM/connector/async_db.hpp, tests/unit/test_async_connector.cpp
IoContext и native async connectors	reactor-style fd readiness и native non-blocking интеграция	lib/include/ORM/async/io_context.hpp, lib/include/ORM/db/connectors/PostgreSQL/postgresql_async.hpp, tests/unit/test_io_context.cpp
Pooling и транзакции	coroutine-aware connection ownership и transactional safety	lib/include/ORM/db/connectors/ThreadSafety/async_thread_safety.hpp, tests/unit/test_async_connector.cpp

6.7. Архитектурен извод за async слоя

Асинхронната архитектура на библиотеката не е просто добавен coroutine wrapper около синхронното ORM ядро. Тя е самостоятелен слой с ясно разграничени отговорности: Task<T> моделира lifecycle-а на корутината, run_on_pool() осигурява универсален offload механизъм, async_db<DB> реализира capability-gated dispatch, IoContext предоставя native readiness integration, а pooling и transaction helper-ите разширяват модела към practically relevant многозадачни сценарии.

Следователно втората архитектурна ос на дипломната работа не се свежда до това „да има async API“. Тя показва как compile-time формализираният access model може да бъде изпълняван през два честно разграничени asynchronous execution path-a — универсален и native — без библиотеката да губи типова дисциплина, lifecycle безопасност или connector transparency. Именно този слой прави възможен следващия анализ на backend

изпълнителните модели.

VII. BACKEND ИЗПЪЛНИТЕЛНИ МОДЕЛИ И NATIVE CLIENT БИБЛИОТЕКИ

Настоящата глава изследва не само как една и съща C++ абстракция се материализира в различни backend-и, но и как се изпълнява през конкретните native client библиотеки. Анализът има две цели. Първо, да покаже, че библиотеката не е ограничена до чисто SQL интерпретация. Второ, да изясни къде асинхронната интеграция може да следва native non-blocking модел и къде честният архитектурен избор остава thread-pool offload върху синхронен драйвер.

След теоретичната рамка от Глава III и архитектурното обяснение на async слой от Глава VI, тук вече не се пита дали различните модели за съхранение са еквивалентни. Вместо това се пита дали една и съща логическа ORM форма може да бъде адаптирана контролирано към тях, без библиотеката да обещава execution semantics, които даден backend не може естествено да изпълнява. Тъкмо в това се проявява стойността на capability-базирания connector contract и на разграничението между универсален async интерфейс и backend-специфична реализация.

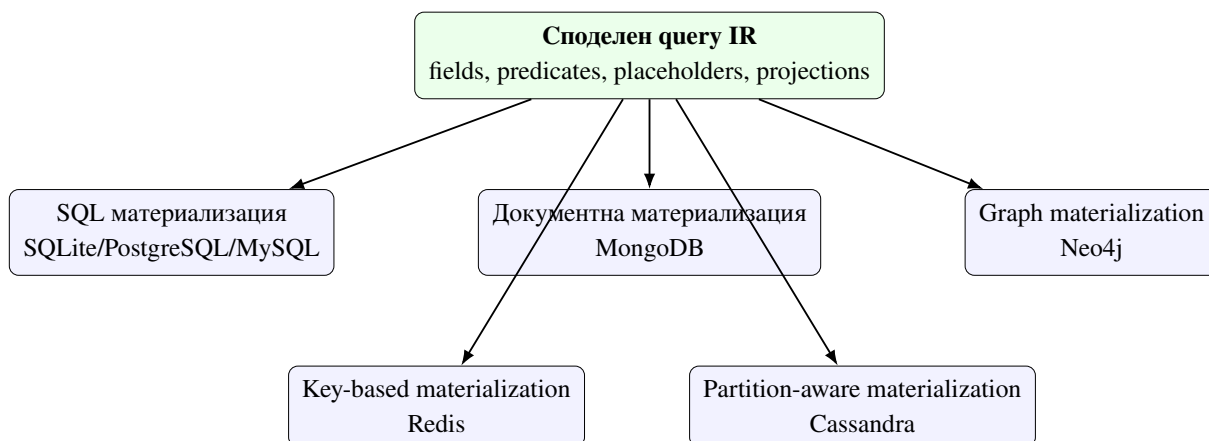
7.1. Методика на анализа

Всеки сценарий следва една и съща последователност: теоретична опора, логическо картографиране от C++ модела, реализация в `connector_trait<DB>`, ограничения и верификация чрез тестове. Това прави сравнението между backend-ите възможно. Не се търси твърдение, че всички системи са еквивалентни, а че една и съща логическа ORM абстракция може да бъде адаптирана контролирано към различни модели за съхранение.

Таблица 11. Еднаква аналитична рамка за всеки backend сценарий

Критерий	Какво се оценява
Теоретична опора	към кой модел за съхранение от Глава III принадлежи backend-ът
Логическо картографиране	как <code>property</code> , <code>field</code> , <code>Rule</code> и <code>table_name_trait</code> се интерпретират в конкретния backend
СарабILITY профил	кои ORM операции са изрично допустими и кои трябва да бъдат отхвърлени
Execution поведение	как се рендират заявки, параметри и резултатни проекции
Тестова опора	кои unit и integration тестове доказват contract-а в хранилището

Тази аналитична рамка е важна, защото позволява сравнението да остане честно. SQL backend-ите не се привилегирват само защото имат по-богат синтаксис, а NoSQL backend-ите не се оценяват като “непълни” само защото отказват операции, които са неподходящи за техния модел на съхранение.



Фигура 13. Един споделен query IR и множество backend-specific materialization пътища

Фигура 13 показва ключовото твърдение на главата: не съществуват множество ORM библиотеки в рамките на хранилището, а една логическа библиотека с различни материализационни пътища. Това е силният аргумент в полза на connector architecture-а — backend разнообразието е овладяно чрез общ IR и explicit contract, а не чрез копиране на API-та.

7.2. Релационен базов сценарий: SQLite

Този сценарий стъпва върху релационния модел, разгледан в Глава III. SQLiteD В е най-естественият референтен backend за библиотеката, защото съчетава реална SQL семантика, малък operational контекст и пряка интеграция чрез sqlite3 API. В него property се материализира като колона, table_name_trait — като име на таблица, а Rule дърветата — като WHERE изрази.

Конекторът декларира supports_joins, supports_transactions и supports_aggregation, което го прави най-близък до класическото ORM очакване. Именно върху него се валидират подготвени заявки, bind логика, find_one() сценарии и result hydration към typed tuples. Интеграционните тестове в tests/integration/test_sqlite.cpp показват не само коректност на SELECT, INSERT, UPDATE и DELETE, но и поведение при transaction-capable connector и join-capable profile.

От академична гледна точка SQLite играе ролята на релационен baseline. Ако дадена ORM форма е проблематична още тук, тя трудно би могла да бъде обобщена към по-екзотични backend-и. Затова настоящата работа използва SQLite като отправна точка, а не просто като един от многото примери.

7.3. Релационна преносимост: PostgreSQL и MySQL

Теоретичната основа и тук е релационният модел, но архитектурният интерес е различен: доколко един и същ query IR остава валиден при различни SQL диалекти и различни правила за bind на параметри. PostgreSQLDB и PostgreSQLLiveDB поддържат supports_joins, supports_transactions и supports_aggregation, а тестовете в tests/unit/test_postgresql_connector.cpp и tests/integration/test_postgresql_live.cpp показват реално изпълнение срещу жива база данни.

MySQLDB и MySQLLiveDB също декларират supports_joins и supports_transactions, но тук е важно различието в bind модела. Докато PostgreSQL може естествено да адресира параметри чрез \$1, \$2, ..., MySQL разчита на позиционни ? placeholders. Това не нарушава общия IR, но показва, че logical portability не означава byte-for-byte идентична backend материализация.

Тъкмо тези SQL family сценарии защитават по-силна теза от “работи със SQL”: библиотеката демонстрира, че query IR-ът е достатъчно общ за повече от един диалект, а

в същото време е достатъчно честен, за да остави нисконивовите различия на connector слоя.

7.4. Документен сценарий: MongoDB

Този сценарий стъпва върху документния модел от теоретичната глава. При MongoDB property се интерпретира като поле в документ, а query predicate дърветата се превеждат към BSON-подобни филтри. select изразите се материализират като projection, а таблицата от ORM гледна точка се превръща в collection. Точно тук се вижда практическата стойност на storage-agnostic query IR: потребителят не описва `\$eq`, `\$and` и projection директно, но конекторът може да ги изведе от същата логическа форма.

Важна подробност е, че `connector_trait<MongoDB>` не декларира SQL-style capability tags. Това не е пропуск, а коректно описание на connector contract-a. Няма претенция, че Mongo backend-ът е join-equivalent на релационните конектори. Вместо това той предлага документно-естествено тълкуване на query IR-a. Unit и integration тестовете в `tests/unit/test_mongodb_connector.cpp` и `tests/integration/test_mongodb_live.cpp` валидират тази адаптация срещу реален driver слой.

Архитектурната стойност на този сценарий е, че доказва едно важно ограничение: една библиотека може да бъде multi-backend, без да твърди, че всички backend-и поддържат едни и същи операции. Именно този тип “контролирана частичност” прави дизайна научно защитим.

7.5. Key-value сценарий: Redis

Този сценарий стъпва върху key-value теорията. При Redis един и същ entity модел може да бъде материализиран по два основни начина: като SET/GET при еднополеви структури или като HSET/HGETALL при многополеви структури. `table_name()` участва в генерирането на key prefix, а достъпът е концентриран около primary-key equality.

Тук ясно се вижда, че споделеният query IR не означава идентична изразителност. RedisDB умишлено не декларира `supports_joins`, `supports_transactions` или `supports_aggregation`. Това ограничение е част от коректния contract, а не слабост на архитектурата. Unit и live тестовете в `tests/unit/test_redis_connector.cpp` и `tests/integration/test_redis_live.cpp` доказват, че библиотеката правилно материализира key-based достъп, без да обещава неподдържани операции.

Този сценарий е особено ценен за дипломната работа, защото принуждава архитектурата да прави ясно разграничение между *логически модел на данните* и *допустим модел на достъп*. Това разграничение е лесно да се забрави при SQL-first библиотеките, но тук е изведено като централен принцип.

7.6. Графов сценарий: Neo4j

Сценарият за Neo4j стъпва върху графовия модел. Entity типът се материализира като label и набор от свойства, а query IR се превежда към MATCH, RETURN и WHERE фрагменти. Това е важен архитектурен тест, защото graph query семантиката е най-отдалечена от класическия SQL модел. Въпреки това логическите елементи — полета, предикати, филтриране и резултатни проекции — остават разпознаваеми.

Neo4jDB и Neo4jLiveDB декларират supports_transactions, но не и join/aggregation capabilities на SQL слоя. Това е смислено: графовият traversal не е същото като SQL join, макар на логическо ниво и двете да свързват данни. Unit и integration тестовете в tests/unit/test_neo4j_connector.cpp и tests/integration/test_neo4j_live.cpp показват, че abstraction-ът може да адаптира един и същ логически модел и към graph backend, ако конекторът поеме отговорността за правилната материализация.

От гледна точка на тезата това е една от най-силните демонстрации, че query IR-ът е семантичен, а не синтактичен слой. Ако IR-ът беше просто недо-SQL, Neo4j сценарият би се разпаднал. Фактът, че е възможна работеща графова материализация, подкрепя централната архитектурна претенция.

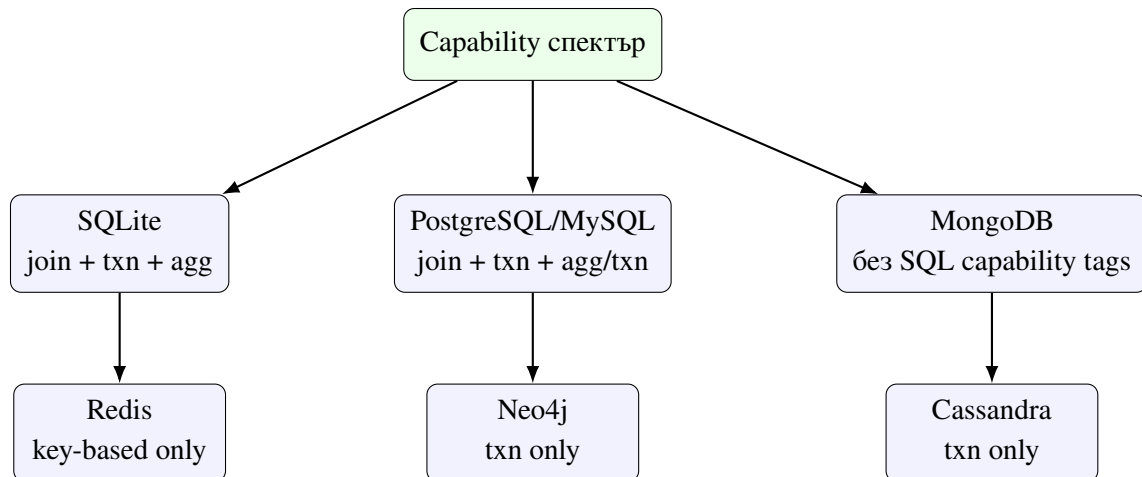
7.7. Wide-column сценарий: Cassandra

Този сценарий стъпва върху wide-column теорията. Макар Cassandra да предлага CQL синтаксис, наподобяващ SQL, действителният модел на достъп е подчинен на partition ключовете и на разпределената организация на данните. Затова библиотеката не може да третира Cassandra като обикновен SQL backend. Наличието на WHERE клауза само по себе си не е достатъчно; тя трябва да бъде съвместима с partition логиката.

CassandraDB и CassandraLiveDB декларират supports_transactions, но не и supports_joins. Това е много показателно. От една страна, backend-ът продължава да е част от същата ORM библиотека. От друга страна, contract-ът категорично отказва операции, които биха били подвеждащи или скъпи в wide-column среда. Unit и live тестовете в

tests/unit/test_cassandra_connector.cpp и tests/integration/test_cassandra_live.cpp дават конкретна верификация на този по-строг operational profile.

Именно поради това Cassandra е най-добрият пример, че разширяемостта на библиотеката не е базирана на фалшиво уеднаквяване, а на формално описани граници на допустимите операции.



Фигура 14. Сравнителен capability профил на backend-ите в хранилището

Фигура 14 синтезира едно от най-важните наблюдения в главата: библиотеката не налага еднакъв capability профил на всички backend-и. Точно обратното — различията се използват като first-class архитектурна информация.

7.8. Синтезирана capability матрица

За да остане сравнението не само качествено, но и таблично проследимо, Фигура 14 може да бъде сведена до компактна capability матрица. Тя не бива да се чете като универсална истина за самите системи за бази данни, а като описание на конкретната архитектурна позиция на библиотеката спрямо тях в разглежданата ревизия на хранилището.

Таблица 12. Capability профили на основните backend-и според наличните тестове и connector specializations

Backend	Joins	Транзак-ции	Агрега-ция	Upsert	Bulk insert	Кратък коментар
SQLite	налична	налична	налична	ограни-чена	ограни-чена	най-близкият пълен релационен reference path в хранилището
PostgreSQL	налична	налична	налична	специ-фична	въз-можна	силен релационен contract с live execution evidence
MySQL	налична	налична	налична	специ-фична	въз-можна	operational path е наличен, но bind и dialect детайлите остават connector-специфични
MongoDB	липсва	липсва	липсва	аналог	не е централна	силен document model без SQL-style capability профил
Redis	липсва	липсва	липсва	аналог	ограни-чена	key-value semantics с минималистичен, но честен contract профил
Neo4j	аналог	налична	липсва	не е централна	не е централна	graph backend с transaction focus, а не със SQL join/aggregation слой
Cassandra	липсва	налична	липсва	не е централна	въз-можна	wide-column backend с ограничен, но формално дефиниран capability набор

Таблица 12 позволява по-прецизно да се види къде библиотеката е най-близо до

класическо ORM поведение и къде съзнателно работи с частичен contract. Релационните backend-и концентрират по-богат capability профил, докато документните, графовите и key-value вариантите се оценяват през други критерии. Именно тази контролирана нееднородност прави multi-backend дизайна инженерно честен.

7.9. Съпоставка между test evidence и backend contract

Таблица 13. Backend-и, capability профил и налична тестова опора

Backend	Capability профил	Unit/contract тест	Integration/live тест
SQLite	joins, transactions, aggregation	capability asserts в tests/integration/test_sqlite.cpp	tests/integration/test_sqlite.cpp
PostgreSQL	joins, transactions, aggregation	tests/unit/test_postgresql_connector.cpp	tests/integration/test_postgresql_live.cpp
MySQL	joins, transactions, aggregation	tests/unit/test_mysql_connector.cpp	tests/integration/test_mysql_live.cpp
MongoDB	без SQL capability tags	tests/unit/test_mongodb_connector.cpp	tests/integration/test_mongodb_live.cpp
Redis	без joins/ transactions/ aggregation	tests/unit/test_redis_connector.cpp	tests/integration/test_redis_live.cpp
Neo4j	transactions	tests/unit/test_neo4j_connector.cpp	tests/integration/test_neo4j_live.cpp
Cassandra	transactions	tests/unit/test_cassandra_connector.cpp	tests/integration/test_cassandra_live.cpp

Таблица 13 е важна, защото превежда сравнението от равнището на narrative analysis към равнището на проверима repository evidence. За всеки backend има поне един ясен contract-level или live-level корелат. Това намалява риска главата да се превърне в

декларативен преглед без техническа опора.

7.10. Сравнителен синтез

Съпоставката между backend-ите показва, че библиотеката успява да запази единен логически модел на равнището на entity типове, полета, placeholder-и и query IR. Различията се появяват на равнището на capability и на физическата организация на данните. Това е и най-важният извод на настоящата глава: успешната multi-backend ORM архитектура не премахва различията между моделите за съхранение, а ги прави контролируеми чрез формален connector contract.

Таблица 14. Съпоставка на backend материализацията

Backend	Основна структура	Логическа интерпретация	Показателно ограничение
SQLite	таблица	пълна SQL ORM интерпретация	стандартна релационна дисциплина и богата query изразителност
PostgreSQL/MySQL	таблица	преносим SQL IR	различен bind модел и диалектни особености
MongoDB	документ	filter/projection интерпретация	частична еквивалентност спрямо SQL операции
Redis	ключ/стойност, hash	key-based entity access	lookup предимно по primary key и липса на join semantics
Neo4j	възел/свойства	graph-pattern интерпретация	graph-специфична query семантика вместо SQL join слой
Cassandra	wide-column row	CQL с ключови ограничения	partition-driven достъп и ограничен predicate profile

В резултат multi-backend design-ът на библиотеката може да бъде защитен с по-прецизна формулировка. Системата не обещава “универсална база данни през един API”, а *унифициран логически слой с експлицитни backend граници*. Именно тази формулировка е едновременно инженерно честна и научно обоснована.

VIII. ВЕРИФИКАЦИЯ И ЕМПИРИЧНА ОЦЕНКА

Тази глава обединява двата вида доказателства, върху които се опира настоящата работа. От една страна стоят `compile-time` ограниченията, `unit` тестовете, `integration` тестовете и `live backend` проверките, чрез които се валидира `correctness` на архитектурата. От друга страна стои емпиричната оценка на асинхронния слой, чиято цел е да покаже кога `coroutine-based` изпълнението носи реална практическа полза и кога неговата цена остава сравнима или по-висока от синхронната алтернатива.

Важният методологичен принцип е, че тези два вида доказателства не се разглеждат като взаимозаменяеми. Тестовата верификация отговаря на въпроса дали библиотеката е коректна спрямо собствените си `contracts` и `backend` ограничения. `Benchmark` оценката отговаря на различен въпрос: носи ли асинхронният `execution` модел измерима полза при натоварвания, в които `waiting` фазата доминира над локалната изчислителна работа. Само съвместното присъствие на двата пласта прави архитектурните претенции на разработката достатъчно убедителни.

8.1. Стратегия на верификацията

Верификационната стратегия следва самата архитектура на библиотеката. `Compile-time` слойът се проверява чрез `concept`-и, `safety` ограничения и типизирано изграждане на `query IR`. `Runtime` слойът се проверява чрез `unit` тестове за рендиране, параметризиране, `thread safety`, миграции, корутинна инфраструктура и асинхронен достъп. Най-външният слой се валидира чрез `integration` тестове с реални `backend`-и, така че да не се смесват `mock` коректност и реално драйверно поведение.

В текущата ревизия на хранилището пълният автоматизиран `suite` включва 271 теста при изпълнение чрез `cctest`. Това число само по себе си не е научен резултат, но е важно като индикатор, че изложените в предходните глави механизми не са оставени на ниво концептуална схема. Те са покрити от систематични проверки по слоеве и по `backend` семейства.

Таблица 15. Основни източници на верификационни доказателства

Източник	Какво валидира	Представителни артефакти
Compile-time ограничения и query IR	типова коректност на заявки, capability-базирано ограничаване и connector contracts	tests/unit/test_query.cpp, tests/ unit/test_postgresql_connector.c pp, tests/unit/test_mongodb_connec tor.cpp
Асинхронна инфраструктура	Task<T>, ThreadPool, I oContext, async_db<DB >, coroutine-aware pooling и транзакции	tests/unit/test_task.cpp, tests/un it/test_thread_pool.cpp, tests/un it/test_io_context.cpp, tests/unit /test_async_connector.cpp
Миграции и thread safety	schema diff логика, rollback semantics, connection guards и конкурентна безопасност	tests/unit/test_schema_migration .cpp, tests/unit/test_thread_safet y.cpp
Live backend интеграции	поведение върху реални SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra среди	tests/integration/test_sqlite.cpp, tests/integration/test_postgresq l_live.cpp, tests/integration/test _mongodb_live.cpp, tests/integrat ion/test_redis_live.cpp
Емпирична оценка	throughput при симулирана waiting фаза и изолирани coroutine overhead измервания	benchmarks/bench_async.cpp

Таблица 15 показва, че доказателствената база е хетерогенна по необходимост. Една compile-time ORM библиотека не може да бъде защитена само с throughput числа, както и асинхронна архитектура не може да бъде защитена само с concept проверки. Изискват се и двата вида доказателства.

8.2. Каталог на unit и integration доказателствата

За да не остане verification стратегията само на равнище обща рамка, тя трябва да бъде разгърната и като конкретен каталог от репозиторийни артефакти. Това е особено важно за настоящата работа, защото архитектурните претенции са distributed между entity

слоя, query IR-a, connector contract-a, async инфраструктурата и live backend проверките. Следващите таблици правят именно това: показват къде в хранилището са разположени основните доказателства и каква роля играе всяка тестова група.

Таблица 16. Основни unit тестове за generic, core и async подсистемите

Файл	Подсистема	Какво валидира
test_types.cpp	базови типови alias-и	коректност на публичните alias-и и базови utility зависимости
test_property.cpp	entity property layer	compile-time модел на скаларните полета и type-trait поведението
test_relationship.cpp	entity relationship layer	infer логика за връзките и storage-strategy поведението
test_query.cpp	query IR и fluent API	коректност на select, insert, update, delete и производните query структури
test_schema_migration.cpp	migration layer	DDL generation, diff логика и state-machine поведение
test_thread_safety.cpp	concurrency helpers	поведение на connection pool, thread-local достъп и transaction guard механизма
test_task.cpp	coroutine task abstraction	lifecycle на Task<T> и basic await/coroutine semantics
test_thread_pool.cpp	worker scheduling	posting, scheduling и worker resource coordination
test_io_context.cpp	async orchestration	event-loop и continuation plumbing за async execution слоя
test_async_connector.cpp	async connector facade	bridge логика между async_db<DB> и coroutine-aware connector hooks
test_wire_protocol.cpp	experimental execution layer	ниско ниво на compile-time/wire-oriented механизми
test_cpp26_reflection.cpp	future language integration	early path за reflection-базирана автоматизация

Таблица 16 е важна, защото показва, че unit слой не се изчерпва с query parser-подобни проверки. Той покрива едновременно логическия модел, execution механиката, миграциите и coroutine-aware инфраструктурата, върху която стъпва асинхронната архитектура.

Таблица 17. Backend-oriented unit тестове и техният contract focus

Файл	Backend фокус	Тип доказателство
test_postgresql_connector.cpp	PostgreSQLDB	is_connector compliance, capability asserts и SQL parameter rendering
test_mysql_connector.cpp	MySQLDB	joins/transactions contract, dialect-specific rendering и bind expectations
test_mongodb_connector.cpp	MongoDB	negative capability validation и BSON-like filter rendering
test_redis_connector.cpp	RedisDB	absence of joins/transactions/aggregation и key-value command materialization
test_cassandra_connector.cpp	CassandraDB	capability ограничения и CQL rendering корелати
test_neo4j_connector.cpp	Neo4jDB	transaction capability и graph-oriented query materialization

Тези backend-oriented unit тестове са критични, защото доказват не само позитивни възможности, но и честно декларирани ограничения. При multi-backend библиотека отрицателната верификация — че даден backend *не* поддържа дадена операция — е също толкова важна, колкото и позитивната.

Таблица 18. Каталог на integration и live тестовите

Файл	Режим на изпълнение	Какво демонстрира
test_mockdb.cpp	винаги наличен integration baseline	query composition, parameter binding, prepared queries и CRUD сценарии без външен backend
test_sqlite.cpp	локален integration тест	реален релационен execution path с capability asserts, joins, transactions и result hydration
test_postgresql_live.cpp	условен live тест	реална връзка, table lifecycle и CRUD изпълнение върху PostgreSQL
test_mysql_live.cpp	условен live тест	реален MySQL driver path, insert/update/delete и select filtering
test_mongodb_live.cpp	условен live тест	изпълнение на document-oriented сценарии срещу жив backend
test_redis_live.cpp	условен live тест	key-value и hash command materialization върху Redis среда
test_cassandra_live.cpp	условен live тест	wide-column execution и backend-specific CQL сценарии
test_neo4j_live.cpp	условен live тест	graph query execution през контролирана Docker среда

Таблица 18 показва, че repository-backed доказателството е разслоено. Не всички backend-и имат еднакъв operational cost за live verification, но хранилището съдържа ясна стратегия как и кога подобни тестове се активират. Това е особено важно за тезата, защото избягва смесването между mock correctness и реално driver-backed поведение.

8.3. Карта на зрелостта според доказателствената опора

Самият capability профил не е достатъчен, ако липсва evidence за реално поведение. Затова е полезно backend-ите да се разгледат и като стълбица на зрелост, определена не само от поддържаните операции, но и от наличната тестова опора. Тук зрелостта означава

комбинация от contract correctness, execution evidence и operational реализъм.

Таблица 19. Backend maturity карта според наличната доказателствена опора

Backend	Ниво на зрелост	Основна доказателствена опора	Интерпретация
MockDB	ниво 3	test_mockdb.cpp	backend-independent execution baseline за query contract-a и prepared query механиката
SQLite	ниво 3 към 4	test_sqlite.cpp	локален реален execution path с богатата query evidence и capability asserts
PostgreSQL	ниво 3	test_postgresql_connector.cpp, test_postgresql_live.cpp	силен contract плюс условна live backend проверка
MySQL	ниво 3	test_mysql_connector.cpp, test_mysql_live.cpp	реален operational path с релационен профил и dialect-specific binding
MongoDB	ниво 3	test_mongodb_connector.cpp, test_mongodb_live.cpp	document-oriented execution evidence без SQL-style capabilities
Redis	ниво 3	test_redis_connector.cpp, test_redis_live.cpp	key-value execution evidence с ограничен, но честен contract профил
Neo4j	ниво 3	test_neo4j_connector.cpp, test_neo4j_live.cpp	graph execution path с Docker-gated live verification
Cassandra	ниво 3	test_cassandra_connector.cpp, test_cassandra_live.cpp	wide-column path с фокус върху формална пригодност и live coverage

Картата в Таблица 19 е важна, защото предотвратява прекалено бинарно четене на backend статуса. Конекторът не е просто наличен или липсващ; той може да бъде contract-correct, execution-validated, partially live-verified или schema-aware богат. Именно този по-нюансиран прочит прави верификационната глава по-научно убедителна и по-близка до реалния статус на хранилището.

8.4. Методология на benchmark оценката

Емпиричната оценка е реализирана чрез Google Benchmark [5] в `benchmarks/bench_async.cpp`. Тя не цели да симулира пълния жизнен цикъл на конкретна база данни, а да изолира архитектурния въпрос дали coroutine-базираното изпълнение освобождава worker нишките по-ефективно от синхронното блокиране, когато между две кратки CPU фази има waiting интервал.

Benchmark наборът е разделен на две групи. Първата група измерва throughput на цели партии от задачи. Синхронният baseline изпълнява CPU работа, след което worker нишката блокира за предварително зададен интервал. Асинхронният вариант използва `SimulatedAsyncIO awaitable`, който спира корутината, планира `delayed resume` чрез `timer queue` и връща продължаването обратно към `ThreadPool`. Втората група измерва изолирани overhead-и: създаване и изчакване на `Task<int>`, `ThreadPool::post()` round-trip и вложени `co_await` вериги.

Методологично е важно, че throughput benchmark-ите използват wall-clock измерване, а не чисто CPU време. Причината е изрично отбелязана и в самия benchmark код: при корутинни сценарии с waiting фаза calibration по CPU време на macOS може да даде подвеждащо поведение, защото suspend-натата задача почти не консумира CPU. Затова съпоставката се извършва чрез `UseRealTime()` и фиксиран брой итерации.

В настоящата глава се използват release резултатите от локалния `build-rel` build tree. Те не трябва да се интерпретират като абсолютни production числа за всички възможни машини и драйвери. Тяхната роля е сравнителна: всички варианти се изпълняват върху една и съща машина, с една и съща параметрична мрежа, така че да се изведе относителният ефект от различния execution модел.

Таблица 20. Семейства benchmark измервания и тяхната цел

Benchmark семејство	Какво измерва	Основни параметри
BM_SyncThroughput	throughput на блокираш baseline, при който worker нишката остава заета по време на waiting фазата	брой нишки, брой задачи, waiting латентност, CPU iterations
BM_AsyncThroughput	throughput на coroutine вариант със suspend/resume и delayed resume чрез timer queue	брой нишки, брой задачи, waiting латентност, CPU iterations
BM_AsyncYieldThroughput	цена на coroutine interleave без изкуствена waiting латентност	брой нишки, брой задачи, CPU iterations
BM_TaskCreateAndWait	минимална цена на създаване и завършване на тривиална задача	единична корутина без външно scheduling
BM_ThreadPoolPost	цена на post () и promise/future round-trip към worker нишка	размер на thread pool-a
BM_NestedCoAwait	overhead на вложено co_await продължаване при различна дълбочина	дълбочина на coroutine веригата

8.5. Основни throughput резултати

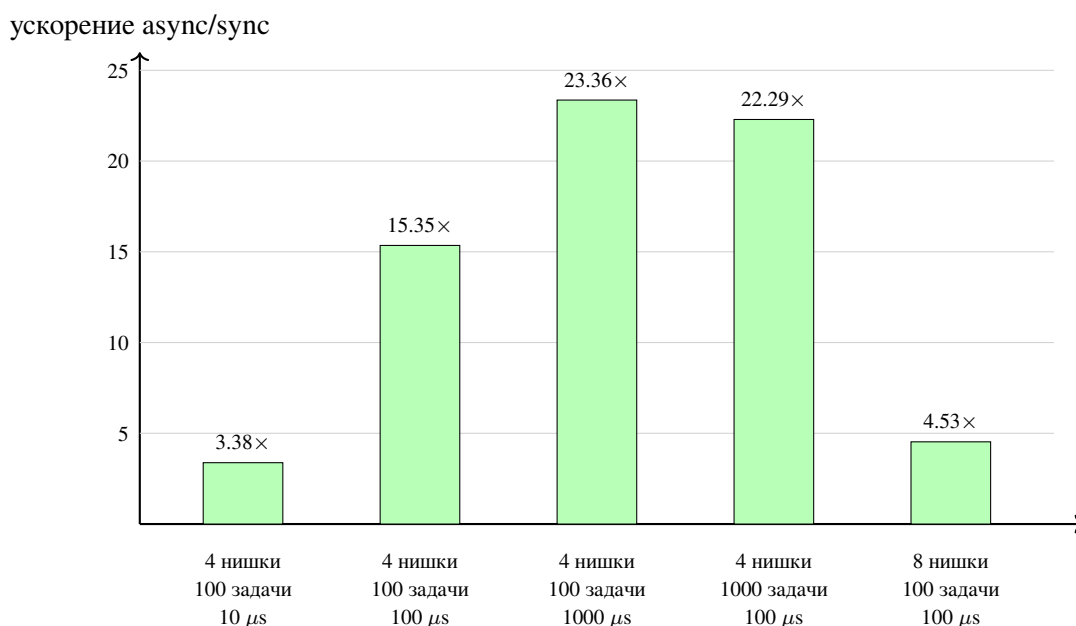
Таблица 21 обобщава представителните release-mode резултати за най-информативните конфигурации. Показани са двойки, за които има директно сравнение между синхронния и асинхронния вариант при едни и същи параметри.

Таблица 21. Representative release-mode throughput results for synchronous and asynchronous execution

Нишки	Задачи	Waiting латентност	Синхронен throughput	Асинхронен throughput	Ускорение
4	100	10 μ s	194.6k ops/s	658.5k ops/s	3.38×
4	100	100 μ s	29.8k ops/s	458.2k ops/s	15.35×
4	100	1000 μ s	3.22k ops/s	75.28k ops/s	23.36×
4	1000	100 μ s	30.08k ops/s	670.5k ops/s	22.29×
8	100	100 μ s	57.13k ops/s	258.5k ops/s	4.53×

Най-важният извод е, че предимството на корутинния модел нараства не когато локалната CPU работа нарасне, а когато waiting фазата започне да доминира над нея. При 4 worker нишки и 100 задачи разликата между 10 и 1000 микросекунди (μ s) waiting латентност увеличава относителното предимство на асинхронния вариант от 3.38× до 23.36×. Това е именно очакваното поведение: при синхронния baseline нишките се изчерпват от блокиране, докато при корутинния модел те се освобождават за друга работа.

Вторият важен извод е, че ефектът не изчезва при по-големи партии. При 4 нишки, 1000 задачи и 100 μ s waiting латентност асинхронният вариант достига 670.5k операции в секунда срещу 30.08k при синхронния baseline. Това показва, че архитектурното предимство не е ограничено до „демо“ случай с малък брой задачи, а се запазва и когато конкуренцията за worker ресурси нарасне.



Фигура 15. Относително ускорение на асинхронния throughput спрямо синхронния baseline при representative I/O-bound конфигурации

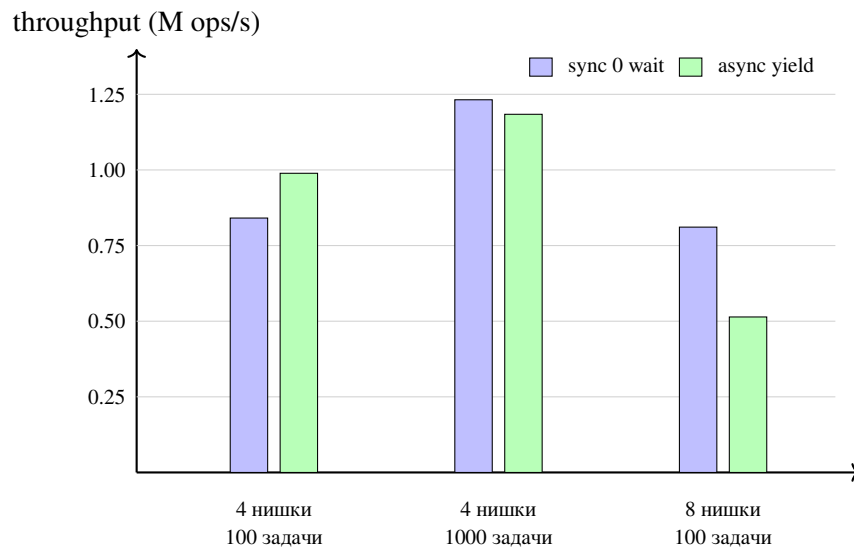
Фигура 15 визуализира същата тенденция в относителна форма. Дори когато броят на worker нишките се удвои до 8, асинхронният вариант остава приблизително 4.5 $\times$ по-бърз при 100 μ s waiting латентност. Това е важен practically relevant резултат: повече нишки намаляват част от натиска върху синхронния baseline, но не елиминират структурното предимство на suspend/resume execution модела.

8.6. Нулева waiting латентност (0ns) и coroutine scheduling overhead

При нулева artificial waiting латентност сравнението престава да измерва освобождаването на worker нишките от I/O изчакване и започва да показва почти чистата цена на suspend/resume механиката. Поради това най-подходящата repository-backed съпоставка комбинира VM_SyncThroughput с нулева waiting стойност и VM_AsyncYieldThroughput, който заменя timer-базирания wait с еднократно PoolYield прехвърляне обратно към ThreadPool.

Таблица 22. Representative release-mode резултати при нулева artificial waiting латентност

Нишки	Задачи	Синхронен throughput	Async yield throughput	Отношение async/sync
4	100	841.0k ops/s	989.4k ops/s	1.18×
4	1000	1.232M ops/s	1.184M ops/s	0.96×
8	100	810.9k ops/s	514.4k ops/s	0.63×



Фигура 16. Throughput съпоставка при нулева artificial waiting латентност

Таблица 22 и Фигура 16 показват, че при липса на waiting фаза корутинният слой вече не получава структурния бонус, наблюдаван в I/O-bound режима. При 4 нишки и 100 задачи yield-only вариантът все още е приблизително 1.18× по-бърз, но при 4 нишки и 1000 задачи той е практически паритетен със синхронния baseline (0.96×), а при 8 нишки и 100 задачи пада до 0.63×. Това е полезен коректив към силните I/O-bound резултати: без външен wait coroutine machinery не „създава“ throughput, а добавя малка, но измерима scheduling цена, която може да бъде компенсирана само частично.

8.7. Microbenchmark интерпретация на overhead-a

Изолираните overhead измервания показват, че корутинната инфраструктура сама по себе си не е доминиращият разход в I/O-bound сценариите. В release build `BM_TaskCreateAndWait` измерва приблизително 71.6 ns за създаване и завършване на тривиална задача. `BM_ThreadPoolPost` остава около 5.0 μs независимо дали pool-ът има 1, 2, 4 или 8 worker

нишки. `VM_NestedCoAwait` достига около 324 ns при дълбочина 10 и около 2.375 μ s дори при дълбочина 100.

Тези числа не трябва да се четат като пряка обещана цена на всяка реална заявка, но са важни с друго. Те показват, че overhead-ът на coroutine machinery е порядъци по-малък от 100–1000 μ s waiting фази, при които асинхронният throughput печели най-много. С други думи, observed speedup-ът не е артефакт на измервателния инструмент, а следствие от това, че worker нишките престават да бъдат заключени в блокиращо изчакване.

8.8. Ограничения на емпиричната оценка

Емпиричната оценка съзнателно използва симулирана waiting фаза, а не пълно end-to-end изпълнение срещу конкретен мрежов сървър за база данни. Това е ограничение, ако целта е да се предскаже абсолютен throughput за реална PostgreSQL или Cassandra инсталация. Но то е и методологично предимство, ако целта е да се изолира execution моделът от влиянието на SQL planner, мрежовите колебания, кеширане в драйвера и storage engine спецификите.

Следователно настоящите benchmark резултати не доказват, че „асинхронното е винаги по-бързо“. Те доказват по-тясна, но по-научно защитима теза: когато архитектурата работи в I/O-bound режим и waiting фазата доминира над локалната CPU работа, coroutine-базираният модел позволява съществено по-добро използване на worker ресурсите от синхронния baseline. Това е точно твърдението, което асинхронният слой на библиотеката трябва да обоснове.

В комбинация с тестовата верификация този резултат затваря двата основни въпроса на дипломната работа. От една страна библиотеката показва формална и тестово подкрепена correctness дисциплина. От друга страна асинхронният execution модел показва измерима practically relevant полза именно в сценария, за който е проектиран. Така емпиричната оценка не стои встрани от архитектурата, а потвърждава нейния централен инженерен и научен смисъл.

IX. НАДГРАЖДАЕМОСТ ЧРЕЗ НОВИ КОНЕКТОРИ

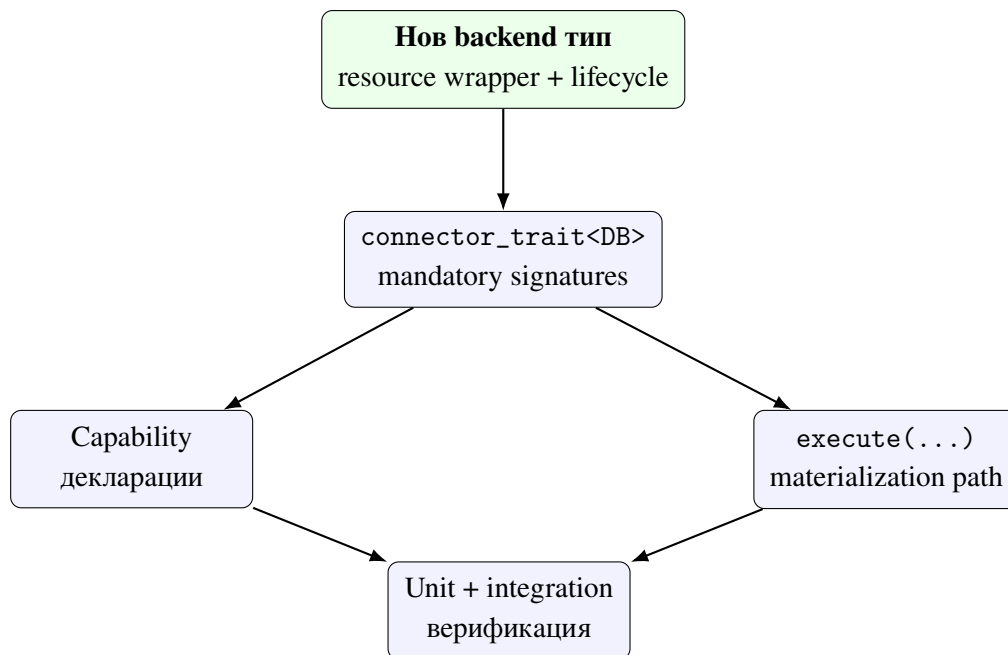
Една от централните претенции на разработката е, че библиотеката е надграждаема без компромис с нейните `compile-time` гаранции. Това изисква не просто наличие на тестове, а ясно дефиниран `connector contract`, който позволява нов `backend` да бъде добавен без промяна в `query API` и без разпадане на типовата дисциплина, `capability` ограниченията и архитектурните инварианти.

В предходните глави беше показано, че библиотеката вече работи с релационни и нерелационни `backend`-и, както и че `async` слой може да бъде реализиран по различен начин според `native` възможностите на конкретния драйвер. Настоящата глава формализира как това е възможно и как трябва да се добавя следващ `backend`. С други думи, тук надграждаемостта престава да бъде общо качество на дизайна и се превръща в методология с конкретни изисквания, `signatures`, `capability` декларации, `async obligations` и критерии за завършеност.

9.1. Формален `contract` на конекторния слой

Конекторът се описва чрез тип `DB` и специализация `connector_trait<DB>`. Това е структурен, а не наследствен `contract`. Няма общ виртуален интерфейс `Connector`. Причината е принципна: различните `backend`-и имат различни възможности и ограничения, които трябва да бъдат видими на `compile-time`. Виртуалната йерархия би изравнила твърде много различия и би прехвърлила част от проверките към `runtime`.

Минималният работещ конектор трябва да осигури `wire_type`, `cursor_type` и подходящи `execute(...)` entry points. Допълнителните способности се означават чрез `capability` tags. Така `contract`-ът остава едновременно строг и разширяем. В тази конструкция няма “магически” `registration` слой: компилаторът вижда специализацията директно и може да провери дали тя удовлетворява `is_connector concept-a`.



Фигура 17. Архитектурен workflow за добавяне на нов connector

Фигура 17 показва, че добавянето на backend не е едноетапно действие. То започва с ясно дефиниран resource wrapper, продължава с trait contract, capability профил и execution materialization, и завършва едва след тестова верификация. Това е съществено за дипломната работа, защото поставя надграждаемостта в дисциплиниран инженерен процес.

9.2. Задължителни елементи на минимален конектор

На практика един backend се счита за минимално интегриран, ако:

1. съществува тип, който представлява backend connection wrapper;
2. съществува специализация `connector_trait<DB>` за този тип;
3. специализацията дефинира `template<typenameT>structwire_type;`
4. специализацията дефинира `cursor_type;`
5. специализацията може да изпълнява поне базовите ORM query форми, които твърди, че поддържа.

Това означава, че “конектор” не е просто рендерираща функция, а пълно описание на начина, по който C++ типовете и query IR-ът преминават към конкретния backend. Ако

липсва дори един от тези елементи, добавеният backend не е архитектурно пълноценен, независимо дали част от заявките могат да бъдат изкуствено изпълнени.

Таблица 23. Задължителни и опционални елементи на конектора

Елемент	Роля
DB тип	wrapper над backend handle, session или mock connection
connector_trait<DB>	централна специализация на contract-a
template<typenameT>\new linestruct\wire_type	преобразува C++ тип към wire представяне
cursor_type	описва начина на обхождане или lifecycle-a на резултатите
execute(...) overload-и	материализират query IR-a към backend поведение
Capability tags	ограничават допустимите операции и lifecycle политики
begin/commit/rollback	необходими при transaction support
ddl_for(...)	необходимо при schema-aware интеграция и migrate<DB>
render_constexpr(...)	опционална compile-time materialization hook за специализирани сценарии

9.3. Типизиране, signatures и именуване

Connector contract-ът е строг и по отношение на типовете. `wire_type<T>::type` е трябва да е стабилна compile-time функция на T. `cursor_type` трябва да моделира реалистичен result iteration или поне ясно да обозначава mock поведение. `execute(...)` overload-ите трябва да приемат `DB&` като първи аргумент и да връщат `orm::result<...>` за SELECT форми или `result<std::tuple<>>` за write операции.

Именуването на конекторите в хранилището показва последователна конвенция. “Локални” или unit-friendly конектори използват имена като SQLiteDB, PostgreSQLDB, MySQLQDB, MongoDB, RedisDB, Neo4jDB и CassandraDB. Реалните live варианти се разграничават експлицитно чрез суфикса LiveDB. Това не е дребен стилев избор, а част от contract clarity: още от името е ясно дали даден тип е mockable adapter, in-memory/local wrapper или реален driver-backed backend.

Липсата на наследяване също трябва да се разглежда като формално правило, а не като `implementation detail`. Нов `connector` не трябва да наследява “обща ORM база”, защото това би смесило backend-специфичната логика с `runtime polymorphism`. Правилният механизъм за разширение е специализацията на `connector_trait<DB>`.

9.4. Carability механизъм и compile-time ограничения

Carability tags са основният начин библиотеката да различава backend-ите по допустими операции. Ако `connector_trait<DB>` не декларира `supports_joins`, тогава опит за `join`-базирана заявка е архитектурно неправилен и следва да бъде отхвърлен още при компилация. Същото важи за транзакции, `aggregation` и `concurrent execution`.

От гледна точка на верификацията това е значимо, защото `correctness` вече не зависи само от тестовете. Част от `contract`-а е проверим директно от компилатора. Тестовете тогава не доказват съществуването на `contract`-а, а че конкретните специализации го прилагат последователно. Именно по тази логика `tests/unit/test_mongodb_connector.cpp` и `tests/unit/test_redis_connector.cpp` валидират отсъствието на `supports_joins`, `supports_transactions` и `supports_aggregation` при съответните `connector`-и.

Тази негативна верификация е също толкова важна, колкото и позитивната. Да докажеш, че backend *не* поддържа дадена операция, е толкова съществено, колкото да докажеш, че поддържа друга.

9.5. Минимален connector skeleton и operational connector skeleton

В хранилището `MockDB` и `MockMigrateDB` са особено ценни, защото играят ролята на `connector archetypes`. Първият показва `operational contract`-а за `query execution`, а вторият — как същият `contract` може да бъде разширен към `schema-aware migration` сценарии.

```
1 template <>
2 struct connector_trait<MockDB>
3 {
4     using supports_joins          = void;
5     using supports_transactions   = void;
6     using supports_aggregation    = void;
7     using supports_upsert         = void;
8     using supports_bulk_insert    = void;
9     using supports_concurrent_execute = void;
10 }
```



```

11     template <typename T>
12     struct wire_type { using type = T; };
13
14     struct cursor_type
15     {
16         [[nodiscard]] bool has_next() const noexcept { return false; }
17     };
18
19     template <typename Response, typename Joins, typename Wheres,
20             typename Limits, typename Groups, typename Orders>
21     static auto execute(MockDB& db,
22         select_query<Response, Joins, Wheres, Limits, Groups, Orders> q)
23         -> result<projected_type<Response>, Response>;
24 };

```

Listing 11: Извадка от connector_trait<MockDB>

Listing 11 е показателен, защото събира в кратка форма почти всички задължителни елементи: capability профил, wire_type, cursor_type и query-specific execute(...) signatures. Освен това connector-ът използва отделни overload-и за select_query, insert_query, update_query и delete_query, което прави mapping-а към backend поведението експлицитен.

```

1  template <>
2  struct connector_trait<MockMigrateDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions    = void;
6      using supports_aggregation     = void;
7      using supports_concurrent_execute = void;
8
9      template <typename T>
10     struct wire_type { using type = T; };
11
12     template <typename... Args>
13     static auto execute(MockMigrateDB& db, Args&&... args)
14     {
15         return connector_trait<MockDB>::execute(
16             static_cast<MockDB&>(db), std::forward<Args>(args)...);
17     }
18
19     [[nodiscard]] static std::string ddl_for(const create_table_op& op);
20     [[nodiscard]] static std::string ddl_for(const add_column_op& op);
21 };

```

Listing 12: Извадка от schema-aware разширението MockMigrateDB

Listing 12 показва как operational connector skeleton-ът може да бъде надграден. Тук

query execution логиката се делегира към MockDB, но се добавят DDL hooks и transaction hooks. Това е добър пример за progressive connector maturity модел.

9.6. Транзакции, нишкова безопасност и prepared queries

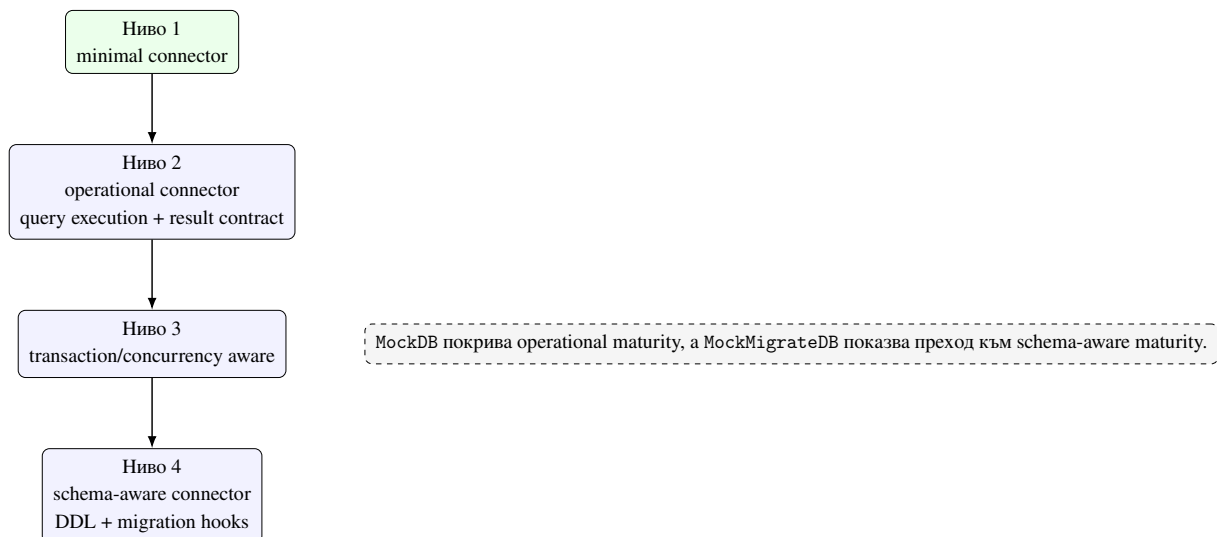
Пълноценният backend contract не се изчерпва с единично изпълнение на SELECT. Ако конекторът декларира supports_transactions, от него се очаква да поддържа begin, commit и rollback, така че transaction_guard<DB> да бъде валиден. Ако декларира supports_concurrent_execute, той трябва да може да участва безопасно в connection_pool<DB,N>.

Prepared query слойът също поставя изискване за стабилност на DB& lifetime и за консистентно bind поведение при многократно изпълнение. Следователно зрелият конектор трябва да бъде оценяван не само по това дали рендира правилен синтаксис, а и по operational contract-а около ресурси, транзакции и повторна употреба. Това е особено важно за backend-и, които имат по-сложен драйверен lifecycle от MockDB.

9.7. Миграции и schema-aware интеграция

Конектор, който участва в migrate<DB>, трябва да предложи и DDL extension points. В хранилището това е демонстрирано чрез MockMigrateDB. Този пример е особено полезен, защото показва как върху вече съществуващ execution backend може да се надгради schema-aware поведение. По този начин библиотеката формализира различни нива на завършеност на конектора.

Тук е важно да се отбележи, че schema-aware поддръжката не е задължителна за всеки backend. Някои конектори могат да бъдат напълно адекватни за query execution анализ, без да поддържат автоматизирани миграции. Но ако конекторът претендира за по-пълна ORM интеграция, наличието на ddl_for(...) hooks и migration тестове вече става задължително.



Фигура 18. Стълба на зрелостта за нов connector

Фигура 18 предлага практичен модел за оценка. Тя е полезна и за самото бъдещо развитие на библиотеката, защото позволява новите backend-и да бъдат позиционирани не бинарно като “има/няма connector”, а според нивото на тяхната интеграция.

9.8. Методика за добавяне на нов backend

Добавянето на нов backend следва ясна последователност. Първо се дефинира backend типът и неговият resource wrapper. След това се реализира `connector_trait<DB>` със задължителните елементи. Следват capability декларации и правилата за рендиране или изпълнение на query IR. Накрая се добавят unit тестове за contract-а и, при възможност, live integration тестове за реалния драйвер.

Архитектурно правилният ред е важен: не се започва от runtime workaround-и, а от формализиране на compile-time границите и на mapping логиката. Ако този ред бъде нарушен, резултатът обикновено е connector, който “работи” за единичен happy path, но не е съвместим с общата архитектура.

Таблица 24. Препоръчителна verification matrix за нов connector

Пласт на верификация	Какво трябва да се докаже	Примерен корелат в хранилището
Concept compliance	is_connector<DB> е удовлетворен	tests/unit/test_mongodb_connector.cpp
Capability honesty	декларираните и недеклаираните capability tags са правилни	tests/unit/test_redis_connector.cpp, tests/integration/test_sqlite.cpp
Execution correctness	query IR-ът се материализира коректно	tests/integration/test_mockdb.cpp и live backend тестове
Resource/lifecycle correctness	transaction и concurrency hooks се държат коректно	tests/unit/test_thread_safety.cpp
Schema-aware correctness	DDL hooks и migration diff-ът работят коректно	tests/unit/test_schema_migration.cpp

9.9. Критерии за „напълно имплементиран и функциониращ“ конектор

На базата на кода и тестовете в хранилището могат да се формулират следните критерии:

1. конекторът удовлетворява is_connector contract-a;
2. capability декларации съответстват на реалното поведение;
3. naming и typing конвенциите са съвместими с останалите конектори;
4. ресурсният lifecycle е коректен и проверим;
5. поддържаните ORM операции са покрити от unit тестове;

6. при наличен live driver има integration тестове срещу реална база данни;
7. при заявена schema-aware поддръжка са налични DDL hooks и migration тестове.

Тези критерии са по-строги от обикновено “има работещ адаптер”, защото изискват консистентност между архитектурно намерение, API contract и верифицирано поведение. Именно това различава ad hoc интеграцията от конектор, който наистина принадлежи към библиотеката.

9.10. Верификация чрез тестове

Тестовият слой в хранилището следва структурата на архитектурата. Unit тестовете за отделните конектори валидират `is_connector compliance`, `capability` декларациите и основното рендиране. `tests/unit/test_thread_safety.cpp` проверява `connection_pool`, `thread_local_db` и `transaction_guard`. `tests/unit/test_schema_migration.cpp` валидира migration diff логиката. `tests/integration/test_mockdb.cpp` и live integration тестовете за отделни backend-и доказват, че abstraction-ът работи и при реално изпълнение.

Следователно верификацията не е концентрирана само на едно място. Тя е разпределена така, че всяко архитектурно твърдение да има поне един пряк тестов корелат. Това превръща connector extensibility-то от декларативна цел в проверима инженерна практика.

9.11. Оценка на наличните конектори

SQLiteDB може да се разглежда като референтна зряла реализация за релационен backend. PostgreSQLDB, MySQLDB, MongoDB, RedisDB, Neo4jDB и CassandraDB вече демонстрират важни части от contract-а и имат различна степен на live покритие. MockDB и MockMigrateDB играят особена роля като верификационни конектори: те не са само тестови двойници, а средство за формализиране на правилата на архитектурата.

Оттук следва и по-общият извод на настоящата глава: надграждаемостта на библиотеката не е добавена впоследствие, а е заложена в самия начин, по който е дефиниран connector слой. Именно това позволява разработката да бъде продължавана с нови backend-и, без да се нарушава формалният характер на compile-time ORM архитектурата.

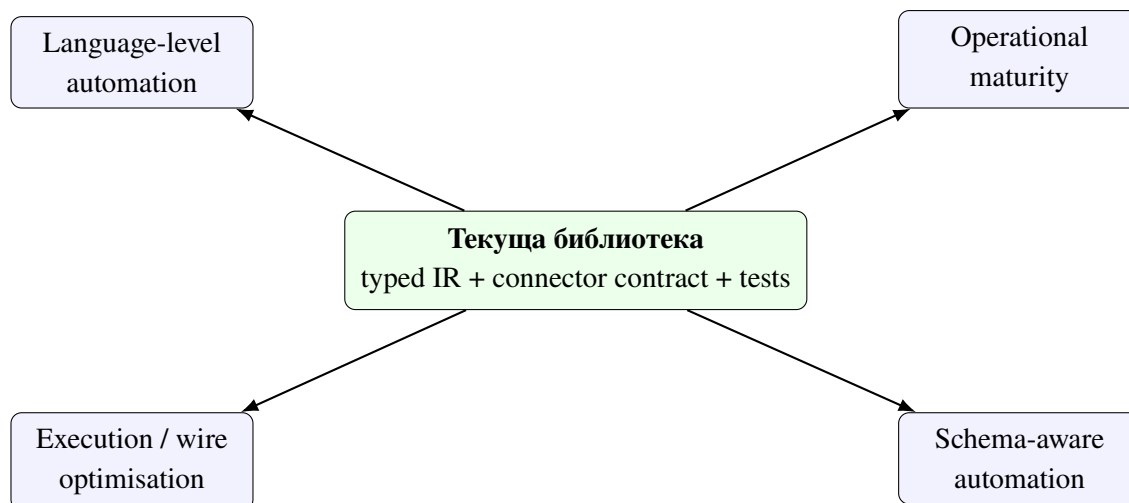
Х. ОГРАНИЧЕНИЯ И БЪДЕЩО РАЗВИТИЕ

Текущата версия на библиотеката вече демонстрира работеща архитектура за Compile-time Object-Relational Mapping върху релационни и нерелационни backend-и, но научната коректност изисква ясно разграничение между вече постигнатото и следващите стъпки. Библиотеката съдържа реален слой за query IR, sarability-базиран конекторен contract, migration прототип, helper-и за нишкова безопасност и набор от unit и integration тестове. Въпреки това част от подсистемите все още трябва да бъдат разширени или валидирани по-задълбочено в production-like условия.

Бъдещото развитие не е произволен списък от идеи. То следва непосредствено от архитектурните решения, анализирани в предходните глави. Ако C++ моделът е първичен носител на логическата схема, следваща естествена стъпка е по-пълна автоматизация на schema-aware tooling. Ако connector слоят е централна точка на extensibility, тогава бъдещата работа трябва да уточни по-фини sarability класове. Ако query IR-ът е реално споделен слой между различни backend-и, тогава той трябва да бъде изпитан при по-богати operational сценарии и performance ограничения.

10.1. Стратегически направления за развитие

От гледна точка на настоящата дипломна работа бъдещото развитие може да бъде групирено в четири стратегически направления. Първото е *по-голяма operational зрялост* на вече наличните backend-и. Второто е *по-пълна автоматизация на схемата и метаданните*. Третото е *намаляване на runtime overhead-a* чрез по-ниско ниво на изпълнение и по-добър контрол върху wire protocol слоя. Четвъртото е *намаляване на boilerplate-a* чрез по-дълбока интеграция с новите езикови механизми на C++.



Фигура 19. Четири стратегически направления за бъдещо развитие

Фигура 19 е полезна, защото показва, че бъдещата работа не е еднопластова. Част от задачите са насочени към експлоатационна зрялост, други — към по-добра compile-time автоматизация, а трети — към performance и runtime инженеринг. Това разграничение е важно, защото различните направления изискват различни критерии за успех.

10.2. Разширяване на live backend покритието

Макар хранилището вече да съдържа live integration тестове за няколко backend-a, зрелостта на отделните реализации не е еднаква. SQLite остава най-естественият референтен релационен път, защото съчетава най-плътно свързани query rendering, bind логика и result hydration. При PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra следващата стъпка е по-широко покриване на operational сценарии: по-богати типове, диагностични пътища при грешка, lifecycle политики за връзки и оценка на поведението при по-сложни заявки.

Това направление е важно не само като инженерно подобрене, а и като научна проверка на границите на общия query IR. Само при по-широко емпирично покритие може да се оцени доколко един и същ логически слой остава адекватен за различни типове системи. Например join-базирана логика, която е естествена за PostgreSQL, има съвсем различен operational смисъл в Neo4j или Cassandra. Следователно live покритието не е просто тестова дисциплина, а средство за изследване на архитектурната приложимост.

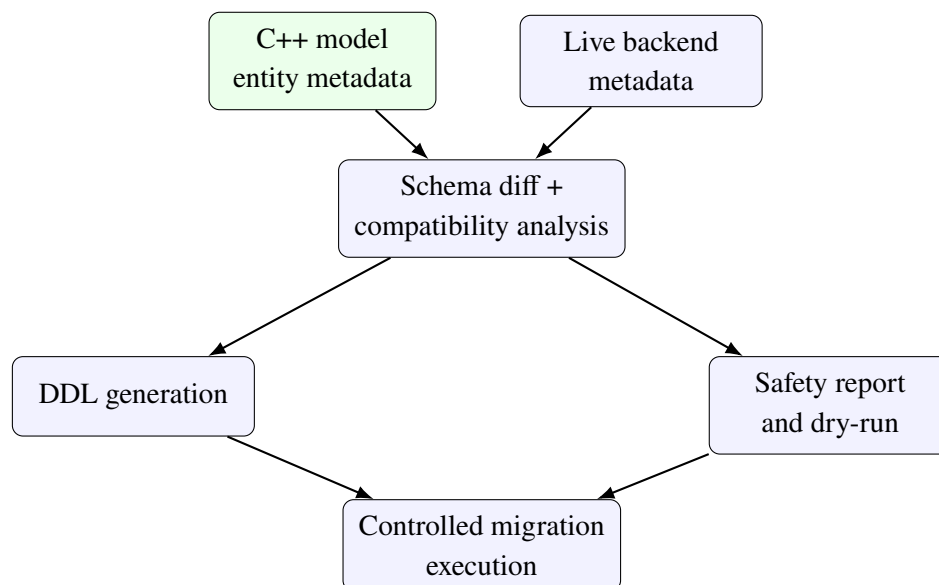
Допълнително, live backend покритието трябва да обхване и негативните пътища: грешки при connection establishment, частично невалидни параметри, driver-specific timeout

сценарии и поведение при прекъсване на връзката. Тези въпроси почти винаги са подценени в ранните ORM библиотеки, но в multi-backend контекст те са решаващи за реалната употреба.

10.3. Пълна schema-aware интеграция и runtime introspection

Прототипният migration слой е достатъчен, за да покаже как C++ entity моделът може да участва в описване на schema evolution. Следващото естествено направление е live introspection на реалната схема, извличане на metadata от драйверите и безопасно изпълнение на DDL операции. Това е особено важно за релационните backend-и, но не е без значение и за документни или wide-column системи, където понятието за schema може да бъде по-гъвкаво или частично.

По-нататъшната работа в тази посока трябва да изясни и границата между compile-time гаранции и runtime факти за вече разположена система. Компиляторът може да валидира query структурата, но не и реалното състояние на production schema без допълнителна runtime информация. Следователно бъдещият migration слой трябва да комбинира два източника на истина: C++ entity описанието и извлечената metadata картина на реалната база.



Фигура 20. Целева посока за schema-aware tooling отвъд текущия migration прототип

Фигура 20 показва, че бъдещият migration pipeline трябва да бъде двупосочен: едновременно да чете от compile-time описанието и от живата система. Това е естествена следваща стъпка, ако библиотеката трябва да бъде използвана не само експериментално,

но и като practical development tool.

10.4. Нишкова безопасност и жизнен цикъл на връзките

Съществуващите helper-и за connection pool, thread-local достъп и transaction guard показват, че библиотеката вече съдържа архитектурна основа за multi-threaded употреба. Бъдещото развитие трябва да отговори на по-конкретни въпроси: как се дефинира `supports_concurrent_execute` за различните драйвери; кога една връзка е безопасна за споделяне; какви са правилата за реконекция, затваряне и отложено освобождаване на ресурси.

Тук има и важен изследователски аспект: различните backend-и имат различна concurrency семантика, а следователно `compile-time capability` обозначаването трябва да бъде достатъчно прецизно, за да не създава подвеждащи обещания. Напълно възможно е бъдещи версии на библиотеката да разделят общото `supports_concurrent_execute` на по-фини `capability` класове като безопасно паралелно четене, паралелно изпълнение с отделни connection handle-и или строга serialization политика.

Практически това означава, че thread-safety helper слой не трябва да остане изолиран помощен модул. Напротив, той трябва да се превърне в архитектурно продължение на connector contract-а и да участва в пълната оценка на зрелостта на даден backend.

10.5. Ниско ниво на изпълнение и wire protocol оптимизации

Експерименталният слой в `WireProtocol/wire_protocol.hpp` показва възможни бъдещи посоки като `io_uring` / `IOCP` awaitable-и, `zero_copy_result`, `batch_insert` и `make_constexpr_sql`. Тези елементи все още не са production-ready комуникационен стек, но очертават възможност библиотеката да се разшири отвъд класическия synchronous driver модел.

Особено перспективни са нулево-копийната обработка на резултати и `compile-time` генерирането на SQL за конектори, които го позволяват. Те биха приближили библиотеката до още по-нисък runtime overhead, без да се жертва типизираната архитектура. Това е важно, защото една честа критика към богато типизираните abstraction слоеве е, че скриват прекомерен runtime разход. Бъдещата работа трябва да покаже, че `compile-time` изразителността и ниският overhead не са задължително противоположни цели.

Освен това performance темата не трябва да се разглежда само през latency или throughput. Тя включва и memory layout на резултатите, броя временни allocation-и, повторната употреба на prepared query артефакти и качеството на driver-level batching стратегиите.

10.6. Интеграция с C++26 reflection и допълнителна автоматизация

Поддржката на C++26 reflection е вече концептуално подготвена чрез detection логика и helper-и, а `tests/unit/test_cpp26_reflection.cpp` показва наличието на early verification path. Публичният API обаче все още използва експлицитни string literal имена за `property<T, Name>`. Бъдещото развитие трябва да превърне reflection path-а в стабилен публичен механизъм, който позволява автоматично извличане на имената на полетата, без това да нарушава обратната съвместимост.

Подобно развитие би намалило boilerplate-а и би направило още по-видима идеята, че C++ моделът е първичният източник на логическата схема. От изследователска гледна точка то би било интересно и защото би доближило библиотеката до model-first стил, при който голяма част от metadata информацията се извлича автоматично, а не се описва повторно.

Тук обаче има и важна граница. Reflection не трябва да разрушава яснотата на compile-time contract-а. Ако автоматизацията намали прозрачността на типа или затрудни съобщенията за грешка, тя може да отслаби една от основните ценности на библиотеката. Следователно тази посока трябва да се развива внимателно.

10.7. Разширяване на connector contract-а и capability таксономията

С добавянето на нови backend-и ще стане необходимо по-прецизно разграничаване между различни класове възможности. Възможно е например бъдещи версии на библиотеката да разграничават отделно graph traversal capability, partial document embedding support, schema introspection support, streaming result support или explicit batching support. Тази посока следва естествено от текущата trait-базирана архитектура.

По-нататъшната работа трябва да пази едно важно правило: новите capability-та не бива да бъдат декоративни. Те трябва да носят реално compile-time значение и да ограничават недопустимите операции. Ако capability tag няма архитектурно последствие, той не добавя стойност. Точно обратното — прави contract-а по-шумен и по-малко точен.

Таблица 25. Приоритетни направления за бъдещо развитие

Направление	Засегнати подсистеми	Очакван ефект	Основно предизвикателство
Live backend maturity	connector-и, integration tests, error paths	по-висока practical reliability	различна operational семантика между backend-ите
Schema-aware automation	migrate<DB>, DDL hooks, metadata introspection	по-близка до production ORM интеграция	съчетаване на compile-time и runtime истина
Execution optimisation	wire protocol layer, prepared queries, batching	по-нисък runtime overhead	запазване на typed abstraction-a
Reflection-driven modelling	entity layer, metadata extraction, API ergonomics	по-малко boilerplate и по-автоматичен model-first подход	баланс между автоматизация и прозрачност
Capability refinement	connector_trait<DB> и static checks	по-точен contract за нови backend-и	избягване на декоративни capability tag-ове

10.8. Необходимост от по-строга емпирична оценка

Една бъдеща версия на работата следва да включва и по-систематичен benchmarking слой. Той трябва да сравнява не само отделни backend-и, но и различни execution режими вътре в библиотеката — директно изпълнение, prepare() + reuse, batch insert сценарии и различни форми на result materialization. Такъв benchmarking би бил ценен както инженерно, така и научно, защото би показал реалната цена на различните архитектурни избори.

Наред с това, полезно би било да се добави и формализирана оценка на error diagnostics. Една от силните страни на compile-time ORM дизайна е именно ранното локализиране на грешки. Следователно качеството на съобщенията за грешка и яснотата на compile-time failure пътищата също са валиден обект на изследване.

10.9. Обобщение

Бъдещото развитие на библиотеката не започва от празно място. Напротив, то стъпва върху вече работеща архитектура, която е достатъчно обща, за да поеме нови backend-и, и достатъчно строга, за да наложи формални ограничения. Именно това прави разработката подходяща както за продължаване като инженерна система, така и за по-нататъшни академични изследвания в областта на compile-time моделирането на достъп до данни.

Най-важното заключение от настоящата глава е, че бъдещата работа не трябва да се разбира като поправка на несъстоятелен прототип. Тя е естествено разгръщане на вече защитима архитектурна основа. Това е съществено различие: библиотеката е достигнала ниво, на което разширението е въпрос на дисциплинирано развитие, а не на концептуално преосмисляне.

XI. ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа представи проектирането, реализацията и анализа на Compile-time Object-Relational Mapping библиотека за C++, ориентирана към работа както с релационни, така и с нерелационни бази данни. В центъра на разработката стои идеята, че query структурата, логическият модел на данните и съществена част от ограниченията върху употребата на backend-ите могат да бъдат описани и валидирани чрез типовата система на езика.

С това работата адресира един конкретен проблем от съвременната практика: разминаването между силно типизиран приложен код и хетерогенните модели за съхранение, които този код трябва да използва. Класическите ORM решения са най-силни в рамките на релационния свят, а чисто driver-oriented подходите често прехвърлят твърде голяма част от коректността към runtime. Настоящата разработка показва, че между тези два полюса съществува жизнеспособна архитектурна позиция — compile-time ORM слой, който е едновременно практичен, формално структуриран и достатъчно честен по отношение на backend различията.

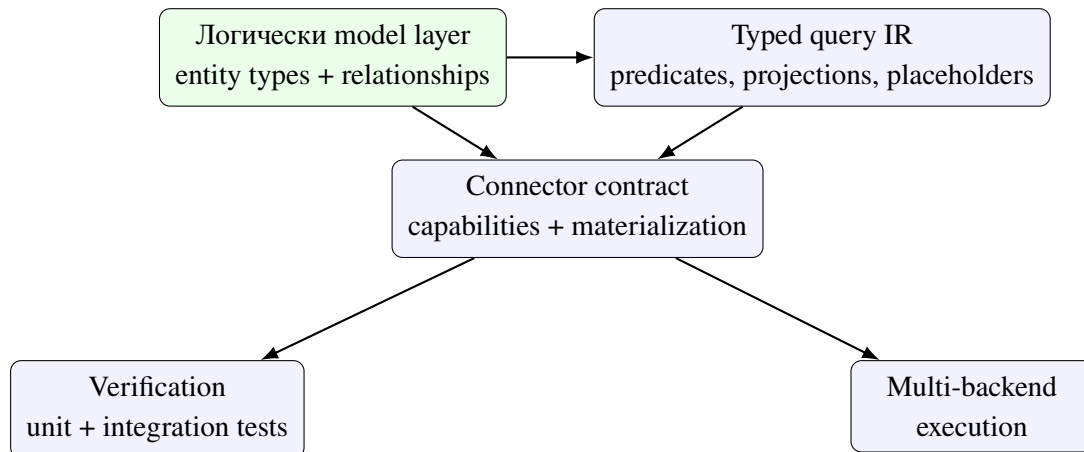
11.1. Обобщение на решената задача

Поставената задача не беше просто да се изгради още един удобен query builder. Целта беше да се проектира библиотека, в която:

1. логическият модел на данните се описва чрез C++ типове;
2. заявките се представят като типизиран IR, а не като низове;
3. backend-ите се интегрират чрез формален connector contract;
4. различията между storage моделите се управляват чрез capability и constraint механизми;
5. коректността се подкрепя едновременно от compile-time проверки и от тестова инфраструктура.

В рамките на дипломната работа всяка от тези подцели беше адресирана с конкретни архитектурни и implementation решения. Entity слойът въведе типизирани

свойства и отношения, политика за съхранение чрез `store_as` и `traits` за именуване на логическите контейнери. Query builder-ът беше организиран около типизиран `intermediate representation`. Connector слойът беше формализиран чрез `connector_trait<DB>`, а verification стратегията бе подкрепена от unit и integration тестове за множество backend-и.



Фигура 21. Синтез на основната конструкция на разработената библиотека

Фигура 21 обобщава цялостната логика на дипломната работа. Тя показва, че библиотеката не е съвкупност от отделни техники, а последователна система, в която model layer, query IR, connector contract, verification и multi-backend execution се поддържат взаимно.

11.2. Обобщение на постигнатите резултати

Разработката показва, че C++ предоставя достатъчно силни механизми — шаблони, `constexpr`, `concepts` и `trait` специализации — за изграждане на ORM слой, в който заявките не се описват като низове. Вместо това те се представят чрез типизиран query IR, който може да бъде анализиран на compile-time и материализиран по различен начин според backend-a.

На равнище модел на данните библиотеката демонстрира как `property`, `relationship`, `store_as` и `table_name_trait` могат да играят ролята на единен логически слой. Този слой не е ограничен до релационната интерпретация, а служи като общо описание на данните и връзките между тях. Именно това позволява една и съща application-level структура да бъде пренасяна към таблици, документи, ключове и стойности, графови възли или wide-column редове.

На равнище архитектура `connector trait`-ът е формализиран като централен

механизъм за разширяемост. Именно чрез него библиотеката адаптира един и същ query IR към SQL, BSON-подобни филтри, Redis команди, Cypher и CQL. Capability механизмът допълва тази архитектура, като прави недопустимите операции видими още при компилация.

На равнище верификация разработката се опира на систематичен набор от unit и integration тестове, които покриват entity слоя, query builder-a, prepared query механизма, миграциите, нишкова безопасност и конкретните конектори. По този начин архитектурните твърдения са подкрепени с реални доказателства от кода и тестовата инфраструктура.

11.3. Научни и приложни приноси

Първият принос на работата е демонстрацията, че practically useful Compile-time Object-Relational Mapping библиотека може да бъде реализирана в стандартен C++. Това се постига без външна кодогенерация, без виртуален connector интерфейс и без принуждаване на потребителя да напуска идиоматичния C++ модел.

Вторият принос е в изясняването на връзката между C++ entity описанието и физическата организация на данните. Показано е, че C++ моделът може да бъде третиран като първичен носител на логическата схема, докато конкретният backend определя начина на материализация. Тази постановка има стойност не само за ORM библиотеките, а и по-общо за системите, които се опитват да комбинират силна статична типизация с хетерогенни източници на данни.

Третият принос е формализирането на connector contract, който обединява разширяемост и строга compile-time дисциплина. Този contract прави възможно надграждането на библиотеката с нови backend-и, без да се разрушават вече съществуващите гаранции в query API-то. Точно тук работата се различава от подходи, които предлагат общ интерфейс, но скриват критичните разлики между backend-ите.

Четвъртият принос е показването, че verification-ът на подобна библиотека трябва да бъде многостепенен. Не е достатъчно само “да има тестове” или само “да има compile-time checks”. Необходима е комбинация от двете: concept-level constraints, capability assertions, query rendering tests, live integration сценарии и migration/thread-safety проверки.

Таблица 26. Връзка между целите на дипломната работа и реализираните резултати

Цел	Реализиран механизъм	Доказателствена опора
Типизиран модел на данните	property, relationship, table_name_trait	entity headers и unit тестовете за property/relationship
Типизиран query слой	query builder-и и typed IR	query headers, tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp
Разширяем multi-backend слой	connector contract и capability tags	connector headers и backend-specific unit/live тестове
Schema-aware посока	migrate<DB> и DDL hooks	tests/unit/test_schema_migration.cpp
Безопасна operational употреба	thread-safety и transaction helper-и	tests/unit/test_thread_safety.cpp

Таблица 26 показва, че дипломната работа не остава на равнището на декларативен design document. За всяка основна цел има конкретен реализиран механизъм и поне един пряк технически корелат в хранилището.

11.4. Основни ограничения и граници на обобщението

Работата също така ясно показва, че multi-backend ORM абстракцията има граници. Не всеки модел за съхранение предлага едни и същи операции, а следователно общият query IR не може да бъде интерпретиран еднакво навсякъде. Именно затова capability и constraint механизмите са толкова важни. Те не отслабват библиотеката, а я правят коректна спрямо различните backend-и.

Допълнително, макар архитектурната основа за миграции, нишкова безопасност и по-ниско ниво на изпълнение вече да съществува, тези подсистеми остават естествено поле за по-нататъшно развитие и емпирично разширяване. Това означава, че работата трябва да бъде оценявана като завършена архитектурна и implementation демонстрация, но не и като последна дума по темата за compile-time достъп до данни.

От академична гледна точка е важно да се подчертае и още една граница. Настоящата разработка не доказва, че `compile-time ORM` е универсално най-добрият подход за всички приложения. Тя доказва нещо по-точно: че за определен клас системи — системи с нужда от ясна статична структура, висока `type safety` и `controlled multi-backend extensibility` — този подход е жизнеспособен и инженерно защитим.

11.5. Значение за практиката и за бъдещите изследвания

Практическата стойност на разработката се състои в това, че библиотеката вече осигурява стабилна основа за по-нататъшно инженерно развитие. `Entity` описанието, `query IR`-ът, `connector contract`-ът и `verification` стратегията са достатъчно ясно формулирани, за да позволят дисциплинирано добавяне на нови `backend`-и и `operational` възможности.

Изследователската стойност е свързана с по-общия въпрос за ролята на типовата система в моделирането на достъпа до данни. Работата показва, че `type system`-ът на `C++` може да служи не само за описание на данни и алгоритми, но и за формализиране на архитектурни граници, допустимост на операции и преход между логически и физически слоеве. Това поставя библиотеката в по-широк контекст, който надхвърля `ORM` темата в тесния смисъл.

11.6. Заключение бележки

Направеният анализ потвърждава, че типовата система на `C++` е достатъчно мощна, за да служи не само за описване на данни и алгоритми, но и за формализиране на достъпа до персистентни системи. Комбинацията от статичен `entity` модел, `query IR`, `trait`-базиран `connector` слой и систематична верификация показва устойчив път към библиотеки, които са едновременно изразителни, безопасни и надграждаеми.

По този начин дипломната работа постига както инженерна, така и изследователска цел: изгражда работеща библиотека и едновременно формулира принципи, които могат да бъдат използвани при бъдещи разработки на `compile-time` слоеве за достъп до данни. Най-същественият извод е, че `compile-time ORM` подходът не е просто теоретично упражнение, а реална инженерна алтернатива за системи, които изискват строгост, яснота и разширяемост при работа с множество модели за съхранение.

ЛІТЕРАТУРА

- [1] Apple Inc. kqueue(2) — kernel event notification mechanism. https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man2/kqueue.2.html, 2009. Manual page.
- [2] Code Synthesis Tools CC. ODB: C++ object persistence framework. <https://www.codesynthesis.com/products/odb/>, 2010. Online documentation.
- [3] DataStax. DataStax C/C++ driver: Futures. <https://datastax.github.io/cpp-driver/2.6/topics/basics/futures/>, 2026. Online documentation, accessed 2026-04-06.
- [4] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [5] Google. Google Benchmark user guide. https://google.github.io/benchmark/user_guide.html, 2026. Online documentation, accessed 2026-04-06.
- [6] D. Richard Hipp. SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine. <https://www.sqlite.org/>, 2000. Online documentation.
- [7] D. Richard Hipp. SQLite C/C++ interface – an introduction. <https://www.sqlite.org/cintro.html>, 2000. Online documentation.
- [8] Maciej Hryniuk and Mateusz Bao. SOCI: The c++ database access library. <https://soci.sourceforge.net/>, 2004. Online documentation.
- [9] ISO/IEC JTC1/SC22/WG21. P1063R0: Core coroutines. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1063r0.pdf>, 2018. ISO/IEC JTC1/SC22/WG21 proposal.
- [10] ISO/IEC JTC1/SC22/WG21. P0912R5: Merge coroutines TS into C++20 working draft. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0912r5.html>, 2019. ISO/IEC JTC1/SC22/WG21 proposal.
- [11] ISO/IEC JTC1/SC22/WG21. ISO/IEC 14882:2020 – programming language C++. International standard, International Organization for Standardization, 2020.
- [12] ISO/IEC JTC1/SC22/WG21. ISO/IEC 14882:2024 – programming language C++. International standard, International Organization for Standardization, 2024.

- [13] Chris Leishman. libneo4j-client — C client library for Neo4j. <https://neo4j-client.net/>, 2026. Online documentation, accessed 2026-04-06.
- [14] MongoDB, Inc. libmongoc: mongoc_client_pool_t. https://mongoc.org/libmongoc/current/mongoc_client_pool_t.html, 2026. Online documentation, accessed 2026-04-06.
- [15] Oracle Corporation. MySQL connector/C – C client library for MySQL. <https://dev.mysql.com/doc/c-api/en/>, 2000. Online documentation.
- [16] Oracle Corporation. MySQL C api developer guide: C api asynchronous interface. <https://dev.mysql.com/doc/c-api/8.4/en/c-api-asynchronous-interface.html>, 2026. Online documentation, accessed 2026-04-06.
- [17] Redis Ltd. and hiredis contributors. hiredis — minimalistic C client library for Redis. <https://github.com/redis/hiredis>, 2026. GitHub repository and documentation, accessed 2026-04-06.
- [18] The PostgreSQL Global Development Group. libpq — C library for PostgreSQL. <https://www.postgresql.org/docs/current/libpq.html>, 1996. Online documentation.
- [19] The PostgreSQL Global Development Group. PostgreSQL documentation: Asynchronous command processing in libpq. <https://www.postgresql.org/docs/current/libpq-async.html>, 2026. Online documentation, accessed 2026-04-06.
- [20] Roland Witt. sqlpp11: A type safe embedded domain specific language for SQL queries and results in C++. <https://github.com/rbock/sqlpp11>, 2014. GitHub repository.